

Web and Data Engineering

Kapitel 3: Frontend-Technologien — HTML, JavaScript und CSS

Prof. Dr. Michael Granitzer

Ziel dieser Vorlesung

Jede Web-Anwendung, die ein Nutzer sieht und bedient, besteht aus drei Technologien: HTML für Struktur, CSS für Darstellung und JavaScript für Verhalten. Diese Vorlesung vermittelt die **Grundlagen** aller drei — nicht als Programmierkurs, sondern als **Architekturwissen** für Web Engineers.

Leitfragen

- Warum ist semantisches HTML mehr als Markup?
- Wie macht JavaScript den Browser zur Laufzeitumgebung?
- Wie erzeugt CSS aus derselben Struktur unterschiedliche Layouts?
- Wie arbeiten HTML, CSS und JavaScript im Browser zusammen?

Die drei Frontend-Technologien haben unterschiedliche Stabilitätshorizonte: HTML-Elemente sind seit HTML5 (2014) weitgehend stabil; CSS erhält laufend neue Features (Container Queries, Cascade Layers, :has()), während die Grundprinzipien — Cascade, Box-Modell, Normal Flow — unverändert bleiben; JavaScript-Frameworks wechseln alle fünf bis acht Jahre den Dominant Player (jQuery → Backbone → Angular → React → Server-Components). Das Architekturwissen — DOM-Modell, Event Loop, Rendering-Pipeline — bleibt davon unberührt und transferiert direkt auf das jeweils aktuelle Werkzeug.

HTML — Struktur und Semantik

Leitfrage: Warum reicht es nicht, eine Web-Seite einfach mit `<div>`-Elementen zusammenzubauen — und was gewinnt man durch semantisches HTML?

HTML wurde 1991 von Tim Berners-Lee am CERN für den Austausch wissenschaftlicher Dokumente entworfen und ist bis heute ein Dokumentformat, keine Programmiersprache. Diese Entscheidung ist kein historischer Zufall, sondern der Grund, warum das Web auf jedem Gerät, in jedem Browser und ohne Laufzeitumgebung funktioniert. Die Schichtung Inhalt → Darstellung → Verhalten ist ein Architekturprinzip: HTML-Inhalte sind ohne CSS lesbar, ohne JavaScript navigierbar. Webseiten, die diese Reihenfolge umkehren und JavaScript zur Voraussetzung machen, verlieren Barrierefreiheit, SEO-Sichtbarkeit und Resilience gegenüber Netzwerkproblemen.

Zwei Versionen derselben Seite

Version A

```
<div class="top">
  <div class="logo">Shop</div>
  <div class="links">
    <div><a href="/">Start</a></div>
    <div><a href="/products">Produkte</a></div>
  </div>
</div>
<div class="middle">
  <div class="left">
    <div class="title">Funkkopfhörer</div>
    <div>Kabelloser Kopfhörer mit...</div>
    <div class="price">€ 79,99</div>
  </div>
  <div class="right">
    <div>Auch interessant: ...</div>
  </div>
</div>
<div class="bottom">© 2026 Shop</div>
```

Version B

```
<header>
  <h1>Shop</h1>
  <nav>
    <a href="/">Start</a>
    <a href="/products">Produkte</a>
  </nav>
</header>
<main>
  <article>
    <h2>Funkkopfhörer</h2>
    <p>Kabelloser Kopfhörer mit...</p>
    <p class="price">€ 79,99</p>
  </article>
  <aside>Auch interessant: ...</aside>
</main>
<footer>© 2026 Shop</footer>
```

Aufgabe: Beide Versionen sehen im Browser identisch aus. Was unterscheidet sie — und für wen?

Für einen sehenden Nutzer mit modernem Browser ist zwischen Version A und B kein visueller Unterschied erkennbar — beide rendern identisch. Der Unterschied wird sichtbar, sobald man die Perspektive wechselt: Ein Screenreader liest `<div>` ohne Kontext, während er `<nav>`, `<article>` und `<footer>` explizit benennt. Eine Suchmaschine kann aus `<article>` + `<h2>` den Hauptinhalt ableiten, aus `<div class="title">` nicht. Ein Browser-Reader-Mode extrahiert `<main>` zuverlässig, `<div class="middle">` nicht. Und ein Entwickler, der nach zwei Jahren zurückkommt, liest semantisches HTML ohne Kommentar — `<div>`-Soup erfordert Reverse Engineering der CSS-Klassen.

Was Version B besser macht

Nutzer	Version A (<div>)	Version B (semantisch)
Screenreader	„div, div, div, Link, div, div..." — keine Orientierung	„Navigation mit 2 Links, Hauptinhalt: Artikel ‚Funkkopfhörer‘, Sidebar"
Suchmaschine	Muss raten, was Hauptinhalt ist	<article> + <h2> = klares Inhaltsranking
Entwickler	Muss CSS-Klassen lesen, um Struktur zu verstehen	Code ist selbstdokumentierend
Browser	Kein Reader-Mode, keine sinnvolle Druckansicht	Reader-Mode extrahiert <article>, Druckansicht setzt <main>

Kernprinzip

HTML trennt

- **Struktur** (was ist ein Absatz, eine Überschrift, eine Navigation?)
- von **Darstellung** (wie sieht es aus? → CSS)
- und **Verhalten** (was passiert bei Klick? → JavaScript).

Zwei der vier Perspektiven haben rechtliche Relevanz: In Deutschland schreibt das Behindertengleichstellungsgesetz (BGG) und die EU-Richtlinie EN 301 549 für öffentliche Einrichtungen barrierefreie Webseiten vor — semantisches HTML ist deren technische Grundlage. Die WCAG-Kriterien 1.3.1 (Info and Relationships) und 2.4.6 (Headings and Labels) sind direkt mit der Wahl zwischen semantischen Elementen und `<div>` verknüpft. Der wirtschaftliche Aspekt: Seiten mit klarer semantischer Struktur erhalten in Googles Qualitätsbewertung höhere Marks für Content-to-Markup-Ratio und erhalten Rich-Snippets häufiger zugewiesen.

Das Technologie-Dreieck des Webs

Jede Web-Anwendung besteht aus drei Schichten mit klarer Verantwortungsteilung:

Schicht	Technologie	Verantwortung	Leitfrage
Struktur	HTML	Was existiert auf der Seite?	Welche Elemente, welche Bedeutung?
Darstellung	CSS	Wie sieht es aus und wo steht es?	Farbe, Layout, Animation, Responsivität
Verhalten	JavaScript	Was passiert bei Interaktion?	DOM-Änderungen, Server-Kommunikation, Zustand

Warum drei getrennte Technologien?

Dasselbe HTML kann mit **verschiedenem CSS** völlig anders aussehen — dieselbe Shop-Seite als Desktop-Layout, als Mobilansicht, als Druckversion. JavaScript kann **DOM und CSSOM verändern**, aber HTML und CSS können auch **ohne JavaScript** funktionieren: Links navigieren, Formulare senden, Hover-Effekte reagieren.

Diese Schichtung ist kein Zufall — sie ist das Architekturprinzip des Webs.

Moderne Komponenten-Frameworks (React, Vue, Svelte) bündeln HTML, CSS und JavaScript in einer einzigen Komponentendatei — was oberflächlich wie eine Verletzung der Trennung aussieht. Konzeptionell ist es eine andere Art der Trennung: nicht nach Technologie, sondern nach fachlicher Verantwortung. Die Trennung innerhalb der Datei bleibt erhalten: Template (Struktur) → Styles (Darstellung) → Script (Verhalten). Dasselbe Prinzip, andere organisatorische Einheit. Die Konsequenzen der Schichten-Unabhängigkeit bleiben: Ein `<a>` navigiert ohne JavaScript, ein `<form>` sendet ohne JavaScript, `:hover`-Effekte reagieren ohne JavaScript.

Hinter den Standards: W3C, WHATWG und die Standardisierung des Webs

Die drei Frontend-Technologien sind keine Produkte einer einzelnen Firma, sondern Ergebnis von Standardisierung.

- **WHATWG** (*Web Hypertext Application Technology Working Group*) ist eine von Browser-Herstellern geprägte Arbeitsgruppe und pflegt heute den **HTML Living Standard** sowie das **DOM**.
- **W3C** (*World Wide Web Consortium*) ist das breitere Standardisierungskonsortium des Webs; dort liegen u. a. **CSS** und **ARIA**.
- **ECMA International** standardisiert **JavaScript** unter dem Namen **ECMAScript**.

Wichtig: HTML lag frueher auch beim W3C. Heute ist die WHATWG die primaere Instanz fuer HTML, waehrend das W3C fuer andere zentrale Webstandards weiter wichtig bleibt.

Browser implementieren diese Spezifikationen. Deshalb funktioniert dieselbe Seite grundsaeztlich in Chrome, Firefox und Safari.

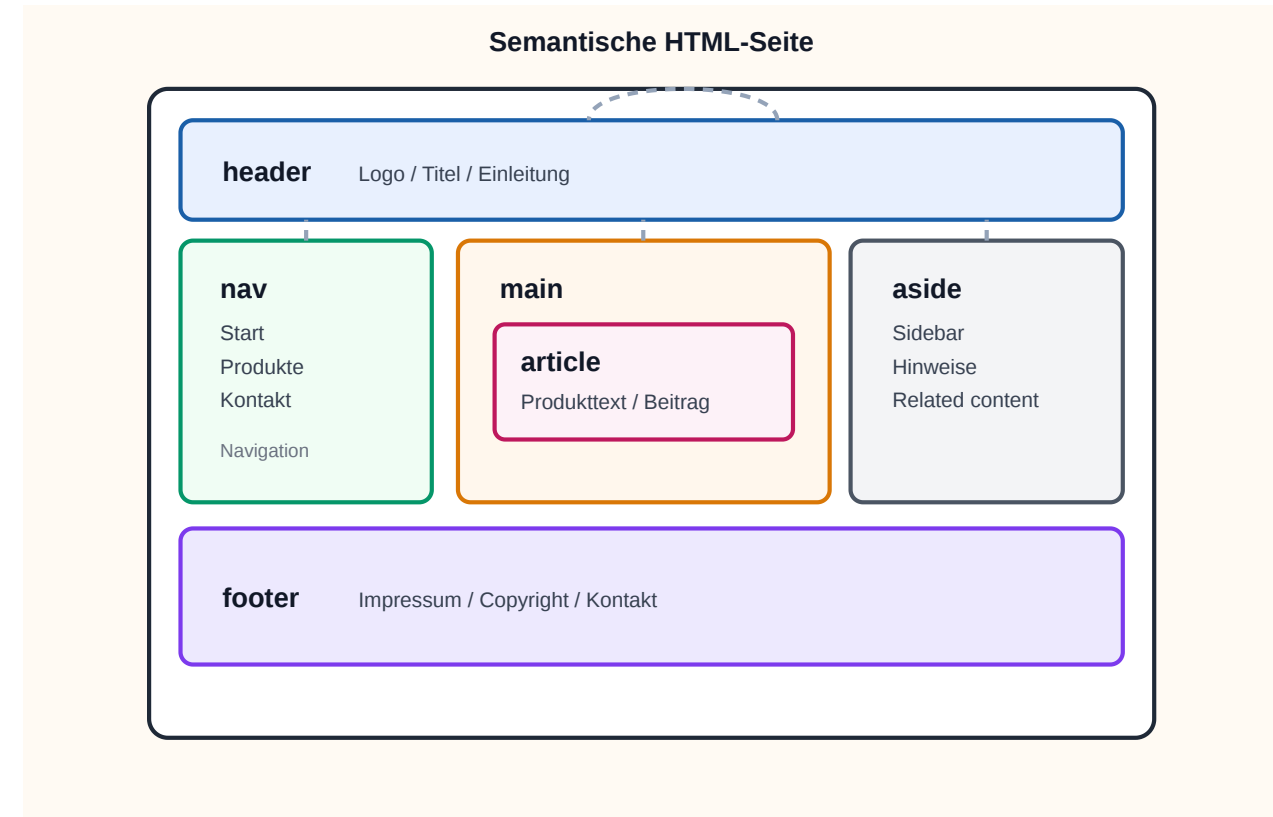


Die Standardisierungslandschaft ist seit 2019 neu geordnet: Die WHATWG hat das W3C als primäre HTML-Standardisierungsinstanz abgelöst, nachdem jahrelange Parallelarbeit zu divergierenden Spezifikationen geführt hatte. Die WHATWG ist keine allgemeine Web-Regulierungsbehörde, sondern eine arbeitsnahe Gruppe rund um Browserhersteller und Editorinnen bzw. Editoren, die HTML und DOM als Living Standards kontinuierlich fortschreibt. Das W3C bleibt dennoch zentral, weil dort viele andere Webstandards organisiert werden, insbesondere CSS über die CSS Working Group und ARIA für Accessibility. ECMA International standardisiert ECMAScript (JavaScript) in jährlichen Releases. MDN Web Docs ist die verlässlichste Querschnittsquelle für Browser-Kompatibilität über alle drei Bereiche; `caniuse.com` hilft bei einzelnen Features vor einer Produktionsentscheidung.

Semantische Elemente in HTML5

HTML5 hat generische Container (<div>,) durch bedeutungstragende Elemente ergänzt:

Element	Bedeutung
<header>	Kopfbereich einer Seite oder Sektion
<nav>	Navigationsbereich
<main>	Hauptinhalt der Seite (einmalig)
<article>	Eigenständiger Inhalt (Blogpost, Produkt)
<section>	Thematischer Abschnitt
<aside>	Ergänzender Inhalt (Sidebar, Hinweis)
<footer>	Fußbereich
<figure> / <figcaption>	Bild mit Beschriftung



Faustregel Wenn ein Element eine **inhaltliche Bedeutung** hat, gibt es dafür ein semantisches HTML-Element. <div> ist für Layout-Container ohne eigene Bedeutung.

HTML5 hat 2014 etwa zwei Dutzend semantische Elemente eingeführt. Die häufig verwechselte Unterscheidung: `<article>` ist eigenständig wiederverwendbar und macht als Fragment außerhalb seiner Seite Sinn (ein Blogpost, ein Produkteintrag, eine Karte in einem Newsfeed); `<section>` ist dagegen nur ein thematischer Abschnitt innerhalb eines größeren Dokuments und ohne diesen Kontext bedeutungslos. `<main>` darf pro Seite nur einmal vorkommen — Browser-Reader-Mode, AMP und mehrere Accessibility-Tools verlassen sich auf diese Einmaligkeit. Screenreader wie NVDA und VoiceOver nutzen Landmark-Elemente (`<nav>`, `<main>`, `<aside>`) für Sprungbefehle, mit denen Keyboard-Nutzer effizient durch die Seitenstruktur navigieren.

Semantik kann mehr: Microformats und Microdata

Semantik in HTML endet nicht bei `<header>` und `<article>`. Für maschinenlesbare Zusatzinformationen gibt es auch Microformats und Microdata.

```
<a class="h-card" href="/team/max">
  <span class="p-name">Max Mustermann</span>
  
</a>

<article itemscope itemtype="https://schema.org/Person">
  <h1 itemprop="name">Max Mustermann</h1>
  <p itemprop="jobTitle">Data Engineer</p>
</article>
```

- **Microformats** nutzen Klassen wie `h-card`, `p-name`, `u-photo`
- **Microdata** nutzt `itemscope`, `itemtype`, `itemprop`
- Beide ergänzen vorhandenes HTML statt es zu ersetzen

Details: [MDN Microformats](#) und [MDN Microdata](#)

Semantik ist nicht binär: `<header>` und `<article>` sind grobe Struktur-Signale; Microdata mit `schema.org`-Vokabular ist feingranular maschinenlesbare Bedeutung. Google's Rich-Result-Snippets (Sterne-Bewertungen, Preise, Veranstaltungstermine direkt in der Trefferliste) basieren auf `schema.org/Product`, `schema.org/Event` und `schema.org/Review-Markup`. Heute ist JSON-LD in einem `<script type="application/ld+json">`-Tag der von Google empfohlene Mechanismus — es bläht das HTML nicht auf und kann server-seitig einfacher generiert werden. Microformats sind dagegen rückläufig; Microdata bleibt ein valider W3C-Standard, wird aber weniger eingesetzt als JSON-LD.

Wofür braucht man das?

Microformats und Microdata sind nützlich, wenn HTML nicht nur für Menschen lesbar sein soll, sondern auch für Programme, die Inhalte erkennen, zusammenfassen oder weiterverarbeiten.

Typische Nutzung

- **Suchmaschinen:** Produkte, Personen, Organisationen, Events, Bewertungen
- **Aggregatoren und Crawler:** Inhalte automatisch aus Seiten extrahieren
- **Interne Systeme:** strukturierte Daten ohne separates JSON-Format
- **Semantic Web / schema.org:** zusätzliche Bedeutung direkt im HTML

Was die Nutzung bringt

- bessere maschinelle Erkennbarkeit von Inhalten
- mögliche Rich Results / strukturierte Vorschau in Suchmaschinen
- weniger Parsing-Hacks in externen Tools
- HTML bleibt die primäre Quelle statt nur ein Container für separate Metadaten

Microformats

- oft für Personen, Kontakte, Termine, Social Links
- leichtgewichtig und direkt lesbar
- Beispiel: h-card, h-event, p-name

Microdata

- häufig für strukturierte Entitäten wie Person, Product, Event
- enger an Vokabulare wie schema.org gebunden
- Beispiel: `itemscope`, `itemtype`, `itemprop`

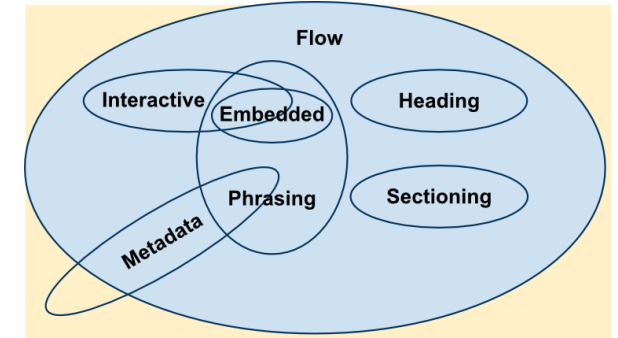
Kurz gesagt: Microformats/Microdata ergänzen Semantik dort, wo reine Textstruktur nicht reicht.

Konkretes Beispiel: Ein Online-Shop, der Produkte mit `schema.org/Product` auszeichnet, bekommt in Google-Trefferlisten oft Sterne, Preis und Verfügbarkeit direkt eingeblendet — was die Click-Through-Rate messbar erhöht. Aggregator-Dienste wie Indeed scrapen Stellenanzeigen automatisch von Firmenseiten, sofern diese `schema.org/JobPosting`-Markup enthalten. Wer also strukturierte Inhalte hat (Produkte, Personen, Termine, Rezensionen), gewinnt durch Microdata reale Sichtbarkeit ohne zusätzlichen Code.

HTML Content Categories

HTML-Elemente gehören zu formalen Kategorien. Diese Kategorien sagen nicht nur, **was** ein Element bedeutet, sondern auch **wo** es im Dokument eingesetzt werden soll.

Kategorie	Idee	Typische Elemente
Flow Content	allgemeine Bausteine im <body>	<div>, <p>, <section>,
Phrasing Content	Inhalt im laufenden Text	, <a>, ,
Heading Content	Überschriftenstruktur	<h1> bis <h6>
Sectioning Content	neue inhaltliche Abschnitte	<article>, <section>, <nav>, <aside>
Embedded Content	eingebettete externe Inhalte	, <video>, <iframe>
Interactive Content	direkt bedienbare Elemente	<a>, <button>, <input>
Metadata Content	Dokument-Metadaten im <head>	<title>, <meta>, <link>



Content Categories sind die formale Seite der HTML-Semantik. Sie helfen dabei zu verstehen, welche Elemente nur im Textfluss vorkommen, welche ganze Abschnitte bilden, welche interaktiv sind und welche ausschließlich Metadaten beschreiben. Das Venn-Diagramm zeigt, dass viele Elemente mehreren Kategorien gleichzeitig angehören koennen. Ein `<a>` ist zum Beispiel Phrasing Content und zugleich Interactive Content.

Warum diese Kategorien wichtig sind

Die wichtigste Regel

Phrasing Content darf in laufendem Text stehen.

Flow Content ist allgemeiner und darf viele Blockelemente enthalten.

Deshalb ist ein `<span>` in einem `<p>` normal, ein `<div>` in einem `<p>` aber **invalides HTML**.

```
<p>Gültig: <span>Inline-Inhalt</span></p>  
<p>Ungültig: <div>Block-Inhalt</div></p>
```

Praktischer Nutzen

- hilft beim **korrekten Verschachteln** von Elementen
- erklärt Validator-Fehler verständlicher
- verhindert stille Browser-Korrekturen
- macht klar, welche Elemente für Text, Abschnitte oder Interaktion gedacht sind

Browser reparieren ungültiges HTML oft stillschweigend. Genau das macht solche Fehler im Layout später schwer nachvollziehbar.

Die wichtigste praktische Regel ist die Unterscheidung zwischen Phrasing Content und Flow Content. Phrasing Content ist alles, was innerhalb eines laufenden Textes sinnvoll ist. Flow Content ist breiter und umfasst auch blockartige Strukturen wie Listen, Abschnitte oder Container. Wenn man ein Flow-Element wie `<div>` in einen Paragraphen setzt, bricht der Browser den Paragraphen implizit ab. Das Ergebnis wirkt oft „fast richtig“, ist aber strukturell falsch und fuehrt zu schwer lesbaren Layout-Bugs.

Formulare: Die Schnittstelle zum Server

Unser Online-Shop braucht ein Registrierungsformular. HTML bietet dafür eingebaute Validierung:

```
<form action="/api/register" method="POST">
  <label for="email">E-Mail:</label>
  <input type="email" id="email"
        name="email" required>

  <label for="password">Passwort:</label>
  <input type="password" id="password"
        name="password" minlength="8" required>

  <button type="submit">Registrieren</button>
</form>
```

Attribut	Wirkung
required	Feld muss ausgefüllt sein
type="email"	Browser prüft E-Mail-Format
minlength	Mindestlänge des Passworts
pattern	Regex-basierte Validierung

Diskussionsfrage: Der Browser validiert clientseitig. Reicht das? Was passiert, wenn jemand das Formular nicht über den Browser, sondern per `curl` oder `Fetch API` abschickt?

Clientseitige Validierung ist ein UX-Feature: Sie liefert sofortiges Feedback ohne Server-Roundtrip und vermeidet unnötige HTTP-Requests für offensichtlich ungültige Eingaben. Sie ist aber trivial zu umgehen — über Browser-DevTools, curl, Postman oder ein selbstgeschriebenes Skript. Jeder Server muss alle Eingaben unabhängig validieren (Defense in Depth). Das gilt unabhängig davon, ob das Frontend eine HTML-Form, eine React-SPA oder eine native App ist: Der Server-Endpunkt ist die einzige verlässliche Sicherheitsgrenze. SQL-Injection (Vorlesung 6) und XSS/CSRF (Vorlesung 7) sind direkte Konsequenzen fehlender serverseitiger Validierung.

HTML5 als Plattform

HTML5 hat den Browser von einem Dokumentbetrachter zu einer **Laufzeitumgebung** erweitert:

Fähigkeit	API / Element	Typischer Einsatz
Multimedia	<audio>, <video>	Streaming, Podcasts
Zeichenfläche	<canvas>, WebGL	Spiele, Datenvisualisierung
Lokaler Speicher	localStorage, sessionStorage, IndexedDB	Offline-Daten, Einstellungen
Hintergrundberechnung	Web Workers	CPU-intensive Aufgaben ohne UI-Blockade
Echtzeitkommunikation	WebSockets	Chat, Live-Updates
Geolocation	Geolocation API	Kartenanwendungen
Push-Benachrichtigungen	Notification API + Service Worker	Progressive Web Apps

Architekturentscheidung: Je mehr Fähigkeiten im Browser verfügbar sind, desto mehr Logik **kann** ins Frontend verlagert werden. **Aber sollte sie das?** Ein Online-Shop, der Preise im Client berechnet, riskiert Manipulation. Ein Offline-fähiger Warenkorb dagegen verbessert die User Experience. Die Grenze zwischen Client und Server ist eine bewusste Architekturentscheidung.

Der moderne Browser ist eine vollständige Anwendungsplattform — vergleichbar mit der Java-Laufzeitumgebung der 2000er oder iOS heute. Progressive Web Apps (PWAs) kombinieren Service Workers (Offline-Cache), Web App Manifest (Installierbarkeit) und Push API (Benachrichtigungen), um native App-ähnliche Erlebnisse ohne App-Store-Distribution zu erreichen. Die strategische Grenzfrage — was gehört in den Browser, was auf den Server, was in eine native App — hängt von Sicherheitsanforderungen (sensible Berechnungen nie im Client), Offline-Anforderungen (Service Worker für Offline-Fähigkeit) und Gerätezugriff (native APIs für Kamera, Bluetooth, NFC nur in nativen Apps vollständig verfügbar) ab.

Responsive Images

Ein Bild sollte nicht auf jedem Gerät gleich ausgeliefert werden. Bei `img` gibt es dafür drei zentrale Attribute:

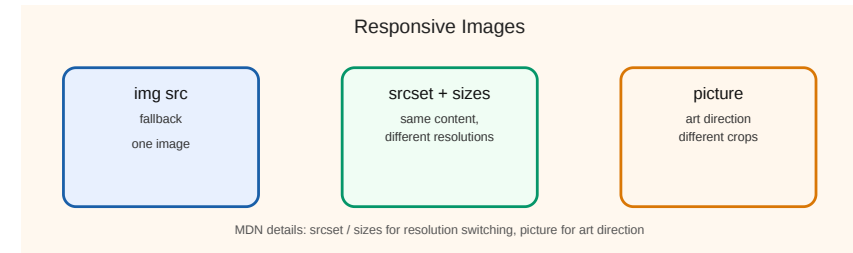
```

```

`src` ist das Fallback-Bild. Wenn der Browser `srcset` oder `sizes` nicht auswertet, lädt er mindestens diese Datei.

`srcset` listet Varianten derselben Bildinformation auf, meist in verschiedenen Breiten: `400w`, `800w`, `1600w`.

`sizes` beschreibt, wie breit das Bild im Layout ungefähr dargestellt wird. Erst damit kann der Browser aus `srcset`



sinnvoll auswählen.

- srcset + sizes = **Resolution Switching**
- Details: [MDN Responsive images](#)

Browser-Logik: sizes sagt, wie viel Platz das Bild im Layout bekommt; srcset sagt, welche Dateibreiten verfügbar sind; der Browser kombiniert beides mit der Pixeldichte des Geräts.

src, srcset und sizes lösen zusammen das Problem, dass ein Bild im Markup zwar nur einmal vorkommt, aber in verschiedenen Layouts unterschiedlich groß erscheint. srcset mit w-Deskriptoren beschreibt reale Quelldateien mit ihren Pixelbreiten. sizes beschreibt dagegen nicht die Dateigröße, sondern die erwartete Darstellungsbreite im Layout, zum Beispiel 100vw auf schmalen Displays und 50vw auf breiten Displays. Der Browser berechnet daraus die benötigte Ressourcenbreite und berücksichtigt zusätzlich die Device Pixel Ratio, also etwa Retina-Displays mit 2x oder 3x Pixeldichte. Wichtig: srcset ist kein CSS-Ersatz. Ohne sizes nimmt der Browser bei w-Deskriptoren häufig 100vw an, was leicht zu übergroßen Downloads führt.

Responsive Images mit <picture>

Wenn sich nicht nur die Auflösung, sondern **der Bildausschnitt oder das Format** ändern soll, reicht `img` allein nicht mehr:

```
<picture>
  <source
    media="(max-width: 768px)"
    srcset="hero-mobile.webp">
  <source
    type="image/avif"
    srcset="hero-desktop.avif">
  
</picture>
```

`<source media="...">` wählt je nach Viewport oder anderen Media Queries unterschiedliche Bildvarianten.

Art Direction bedeutet: dieselbe Information wird für verschiedene Kontexte anders zugeschnitten oder komponiert, etwa engerer Crop auf Mobile.

Formatwahl bedeutet: der Browser kann moderne Formate wie AVIF oder WebP bevorzugen, wenn er sie unterstützt.

Das innere `<img>` bleibt Pflicht: Es ist Fallback, liefert das `alt`-Attribut und wird

- <picture> = **Art Direction** oder Format-Auswahl
- img[srcset][sizes] = **gleiche Bildinformation**, nur andere Auflösung
- Details: [MDN <picture>](#)

<picture> erweitert img um eine Auswahlstufe vor dem eigentlichen Laden des Bildes. Der Browser prüft die source-Elemente der Reihe nach und verwendet die erste passende Kombination aus media-Bedingung und optionalem type. Das ist didaktisch wichtig, weil hier zwei verschiedene Ziele unterschieden werden müssen. Erstens **Art Direction**: Für Mobile wird oft ein anderer Ausschnitt benötigt, weil ein breites Banner auf schmalem Bildschirm inhaltlich unlesbar wird. Zweitens **Formatverhandlung**: Ein Browser, der AVIF unterstützt, kann eine kleinere Datei laden; ältere Browser fallen auf JPEG oder PNG zurück. <picture> ersetzt srcset nicht, sondern ergänzt es: Innerhalb eines source kann wiederum srcset stehen, um für den gewählten Ausschnitt mehrere Auflösungen anzubieten.

Kann HTML das allein? Progressive Enhancement

Bevor man CSS oder JavaScript einsetzt, lohnt die Frage: Was kann HTML nativ?

Anforderung	HTML allein?	Erfordert CSS?	Erfordert JS?
Navigation zwischen Seiten	<code><a href></code> — ja	—	—
Formulardaten an Server senden	<code><form action></code> — ja	—	—
E-Mail-Format validieren	<code><input type="email"></code> — ja	—	—
Bild mit Beschriftung	<code><figure></code> + <code><figcaption></code> — ja	—	—
Dreispaltiges Layout	nein	Grid/Flexbox	—
Hover-Effekt auf Button	nein	<code>:hover</code> Pseudo-Klasse	—
Daten vom Server nachladen ohne Seitenreload	nein	nein	<code>fetch()</code>
Drag & Drop im Warenkorb	nein	nein	Event Handling

Progressive Enhancement Beginne mit funktionierendem HTML. Füge CSS für Darstellung hinzu. Ergänze JavaScript nur wo nötig. Jede Schicht **erweitert**, statt die vorherige zu ersetzen.

Progressive Enhancement ist der Gegenentwurf zu „JavaScript-First-Sites“: Anstatt JavaScript als Voraussetzung anzunehmen und HTML als Beiwerk zu behandeln, wird die Funktionalität *zuerst* in HTML modelliert. Konsequenz: Die Seite funktioniert auf jeder Plattform, mit jedem Browser, ohne JavaScript — sie ist nur weniger komfortabel. Mit JS-Aktivierung kommen Verbesserungen wie Live-Validierung, Inline-Updates, Animationen. Das Gegenmodell — *Graceful Degradation* — geht den umgekehrten Weg: Es startet mit der vollen JS-Erfahrung und versucht, sie für ältere Browser herunterzubrechen. Beides ist legitim, aber Progressive Enhancement ist robuster, weil HTML der stabilere Baseline ist.

Takeaway

HTML definiert die Struktur und Semantik von Web-Inhalten. Der Unterschied zwischen `<div>`-Suppe und semantischem HTML ist nicht kosmetisch — er bestimmt Barrierefreiheit, Auffindbarkeit und Wartbarkeit. Mit HTML5-APIs wird der Browser zur Plattform, was die Frage aufwirft, wie viel Logik ins Frontend gehört. HTML ist deshalb keine reine Markup-Sprache, sondern die Grundlage der Frontend-Architektur.

HTML ist das einzige Frontend-Format, das ohne Laufzeitumgebung verarbeitet wird: Suchmaschinen, Screenreader, RSS-Reader, AI-Crawler und Archivierungssysteme (Wayback Machine) lesen HTML direkt. CSS und JavaScript sind für diese Konsumenten optional oder inexistent. Semantisches HTML ist daher kein Komfortmerkmal, sondern die Interoperabilitätsschicht des Webs — der einzige gemeinsame Nenner aller Verarbeitungspipelines, die auf Web-Inhalte zugreifen.

JavaScript — Die Sprache des Webs

Leitfrage: JavaScript ist die einzige Programmiersprache, die nativ in jedem Browser läuft. Was muss man über sie wissen, um Web-Systeme architektonisch zu verstehen?

JavaScript wurde 1995 von Brendan Eich in zehn Tagen entworfen und hieß ursprünglich Mocha, dann LiveScript, dann JavaScript — ein Marketingentscheid von Netscape, um von Suns Java-Popularität zu profitieren. Technisch sind JavaScript und Java vollständig unverwandt. Das Ausführungsmodell — ein einziger Call Stack, eine Event Queue, ein Event Loop — ist der Grund, warum JavaScript trotz Single-Thread nicht blockiert: I/O-Operationen (fetch, setTimeout, DOM-Events) werden an Browser-APIs delegiert und kehren als Callbacks/Promises in die Queue zurück. Dieses Modell ist identisch in Node.js, weshalb Node denselben Concurrency-Stil auf dem Server ermöglicht.

Der DOM: HTML als programmierbare Struktur

Der Browser parst HTML und erzeugt den **Document Object Model (DOM)** — einen Baum, den JavaScript lesen und verändern kann:

```
document
├── html
│   ├── head
│   └── body
│       ├── h2 "Funkkopfhörer"
│       ├── p  "€ 79,99"
│       └── button "In den Warenkorb"
```

Produktseite unseres Online-Shops

```
// Preis aus dem DOM lesen
const price = document
  .querySelector('.price')
  .textContent;

// Badge aktualisieren
const badge = document
  .querySelector('.cart-badge');
badge.textContent = '3';

// Neues Element erzeugen
const msg = document.createElement('p');
msg.textContent = 'Hinzugefügt!';
document.body.appendChild(msg);
```

Kernidee

Der DOM ist die **Brücke zwischen HTML und JavaScript**. Jede Änderung am DOM wird sofort im Browser sichtbar.

Der DOM ist nicht das HTML-Dokument — er ist der interne Baum, den der Browser aus dem geparsten HTML aufbaut. „View Source“ im Browser zeigt das ursprüngliche HTML-Dokument; der DevTools-Inspector zeigt den aktuellen DOM-Zustand, der durch JavaScript beliebig verändert sein kann. Bei einer Single-Page-App können diese beiden vollständig auseinanderlaufen: Der Quelltext enthält nur ein leeres `<div id="root">`, der DOM enthält die vollständige gerenderte Anwendung. Das ist der Kern des SEO-Problems mit clientseitigem Rendering: Crawler, die kein JavaScript ausführen, sehen den leeren Quelltext, nicht den gerenderten DOM. Server-Side Rendering (SSR) löst dieses Problem, indem der initiale DOM serverseitig erzeugt und als vollständiges HTML ausgeliefert wird.

JavaScript als Brücke zwischen HTML und CSS

JavaScript ist die **einzige** Frontend-Technologie, die sowohl den DOM (HTML) als auch das CSSOM (CSS) verändern kann. Das macht JS zum **Orchestrator**:

CSSOM bedeutet *CSS Object Model*: der interne Baum der CSS-Regeln, den der Browser aus Stylesheets und Inline-Styles aufbaut. Wenn JavaScript Klassen, Inline-Styles oder Custom Properties ändert, beeinflusst es damit nicht nur HTML, sondern auch die berechnete Darstellung.

```
// JS → HTML: DOM-Struktur ändern
const msg = document.createElement('div');
msg.textContent = 'Hinzugefügt!';
document.body.appendChild(msg);

// JS → CSS: Klassen steuern Darstellung
button.classList.add('loading');      // CSS-Transition startet
button.classList.remove('loading');    // CSS-Transition endet

// JS → CSS: Custom Properties dynamisch setzen
document.documentElement.style.setProperty('--cart-count', '3');
```

Die Richtung der Abhängigkeit

```
HTML  ←— JavaScript  —→ CSS
(DOM)   liest & ändert   (CSSOM)
```



- **HTML → JS:** HTML existiert ohne JS (Links, Formulare funktionieren). JS **erweitert** HTML.
- **CSS → JS:** CSS reagiert auf Zustände (:hover, :checked) ohne JS. JS **triggert** CSS-Übergänge über Klassen.
- **JS → beide:** Nur JS kann DOM und CSSOM gleichzeitig manipulieren — deshalb ist JS die Verhaltensschicht.

Konsequenz: Wann immer ein visueller Effekt auch per CSS lösbar ist (Transition, Animation, Hover), sollte CSS es übernehmen — JS steuert nur den **Auslöser** (Klasse setzen), nicht die **Animation** selbst.

Diese Aufteilung ist nicht nur Stil — sie ist auch Performance. CSS-Transitions und -Animations laufen im Compositor-Thread des Browsers, also *außerhalb* des JavaScript-Hauptthreads. Eine in CSS animierte Karte bleibt flüssig, auch wenn JavaScript gerade beschäftigt ist. Eine in JavaScript animierte Karte (mit `requestAnimationFrame` und manuellem Setzen von `style.left`) ruckelt, sobald JS andere Arbeit hat. Diese architektonische Trennung „JS triggert, CSS animiert“ ist die Grundlage für 60-FPS-Webseiten.

Events: Auf Nutzeraktionen reagieren

JavaScript arbeitet **ereignisgesteuert**: Code wird ausgeführt, wenn etwas passiert. In unserem Online-Shop:

```
const button = document.querySelector('#add-to-cart');

button.addEventListener('click', async (event) => {
  // 1. Sofortiges visuelles Feedback
  button.textContent = 'Wird hinzugefügt...';
  button.disabled = true;

  // 2. Server kontaktieren (asynchron)
  const response = await fetch('/api/cart', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ productId: 42, quantity: 1 })
  });

  // 3. UI aktualisieren basierend auf Serverantwort
  if (response.status === 201) {
    button.textContent = '✓ Im Warenkorb';
    updateCartBadge();
  }
});
```

Diskussionsfrage: Der Button wird sofort deaktiviert und der Text geändert, **bevor** die Serverantwort eintrifft. Warum macht man das? Was passiert, wenn die Netzwerkverbindung abbricht?

Das **optimistische UI-Update** ist ein etabliertes Muster für wahrgenommene Performance: Statt auf die Serverantwort zu warten (typisch 100–500ms Netzwerklatenz), aktualisiert der Client sofort seinen lokalen Zustand und zeigt das Feedback. Wenn der Server bestätigt, gibt es nichts zu tun. Wenn der Server fehlschlägt, muss der lokale Zustand zurückgerollt werden — der Button-Text wird zurückgesetzt, eine Fehlermeldung erscheint. Das Rollback ist die schwierigere Hälfte: Der Client muss den vorherigen Zustand zwischengespeichert haben und bei Fehler exakt wiederherstellen. Bibliotheken wie React Query oder SWR stellen dafür explizite `onMutate/onError`-Hooks bereit, die das Rollback strukturiert handhaben.

Fetch API: Die Client-Seite von HTTP

Die Fetch API ist die Brücke zwischen Frontend (Vorlesung 03) und Protokoll (Vorlesung 04):

```
// GET: Produktliste laden
const response = await fetch('/api/products?category=electronics&sort=-price');
const products = await response.json();

// POST: Bestellung aufgeben
const result = await fetch('/api/orders', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ productId: 42, quantity: 1 })
});
if (result.status === 201) {
  const order = await result.json();
  window.location.href = `/orders/${order.id}`; // Weiterleitung
}
```

Fetch-Konzept	HTTP-Gegenstück
method: 'POST'	HTTP-Methode — definiert die Absicht
headers	HTTP Request Headers — Metadaten
response.status	HTTP-Statuscode — Ergebnis der Operation
response.json()	Body parsen — die Repräsentation der Ressource

Die Fetch API ist die moderne Schnittstelle zwischen JavaScript und HTTP — sie hat den älteren XMLHttpRequest (XHR) abgelöst. Konzeptionell wichtig: `fetch()` gibt ein **Promise** zurück, das sich auflöst, sobald die *Header* der Antwort eingetroffen sind — nicht der vollständige Body. `response.json()` ist ein zweiter Schritt, der den Body parst und ebenfalls ein Promise zurückgibt. Das mag umständlich wirken, ermöglicht aber Streaming: Bei einer großen Antwort können Sie schon mit den Headern arbeiten (z.B. Status oder Content-Type prüfen), bevor der gesamte Body geladen ist. Diese Tabelle zeigt explizit, wie 1:1 die Fetch-API-Felder auf HTTP-Konzepte abbilden — eine Brücke zur nächsten Vorlesung.

Was kann schiefgehen?

Aufgabe: Dieser Code funktioniert im Normalfall. Welche Probleme können auftreten — und wie sollte man sie behandeln?

```
const response = await fetch('/api/products/42');  
const product = await response.json();  
document.querySelector('.name').textContent = product.name;  
document.querySelector('.price').textContent = product.price + ' €';
```

Die fünf Fehlerfälle dieses Code-Snippets decken das gesamte Spektrum netzwerkbasierter Fehlerquellen ab: (1) **Netzwerk nicht erreichbar** — `fetch` wirft eine Exception, kein Response-Objekt entsteht. (2) **HTTP-Fehler (4xx/5xx)** — `fetch` liefert ein Response-Objekt mit `response.ok === false`; `.json()` parst den Fehler-Body, der häufig HTML statt JSON enthält. (3) **Langsame Verbindung** — UI zeigt veraltete oder keine Daten, kein Feedback für den Nutzer. (4) **Kein JSON im Body** — `.json()` wirft eine `SyntaxError-Exception`. (5) **Fehlendes DOM-Element** — `querySelector` gibt `null` zurück, Zugriff auf `.textContent` löst `TypeError` aus. Der Unterschied zwischen Code der nur im Normalfall funktioniert und robustem Produktivcode liegt ausschließlich in der Behandlung dieser Randfälle.

Probleme und Lösungen

Problem	Was passiert	Lösung
Server nicht erreichbar	fetch wirft Exception	try/catch + Fehlermeldung anzeigen
404 Not Found	response.ok ist false, .json() liefert Fehler-Body	Status prüfen vor .json()
Langsame Verbindung	UI zeigt alte/keine Daten	Loading-Indicator anzeigen
Response ist kein JSON	.json() wirft Exception	Content-Type prüfen
DOM-Element existiert nicht	querySelector liefert null → Fehler	Null-Check oder optionale Verkettung (?.)

Robuste Fehlerbehandlung ist keine Kür — sie ist der Unterschied zwischen einem Prototyp und einem Produktivsystem.

Eine subtile Falle bei Fetch: `fetch()` wirft *nur* dann eine Exception, wenn das Netzwerk komplett versagt (kein DNS, keine Verbindung). HTTP-Statuscodes wie 404 oder 500 gelten technisch als „erfolgreich übertragen“ — `fetch` liefert ein gültiges Response-Objekt, und `response.ok` ist `false`. Wer das nicht prüft, behandelt Fehlerseiten wie gültige Daten und versucht, HTML-Errorpages als JSON zu parsen. Das ist eine der häufigsten Anfänger-Fallen mit der Fetch API. Robuste Patterns prüfen immer `if (!response.ok) throw ...` vor dem `.json()`-Aufruf.

Warum Frameworks existieren: Das Skalierungsproblem

Unser Warenkorb-Button funktioniert mit `querySelector` und `addEventListener`. Was passiert, wenn die Anwendung wächst?

```
// 5 Zeilen für einen Button – übersichtlich
button.addEventListener('click', () => { badge.textContent = '3'; });

// Aber: 50 Produkte, Filter, Sortierung, Pagination, Warenkorb-Sidebar...
// → Hunderte querySelector-Aufrufe, manuelles DOM-Tracking, Zustandschaos
```

Das Problem imperativer DOM-Manipulation

Komplexität	Vanilla JS	Framework (React/Vue)
Ein Button	Einfach, direkt	Overhead — zu viel Abstraktion
10 interaktive Elemente	Machbar, wird unübersichtlich	Komponenten helfen
Komplexe SPA (Shop, Dashboard)	Zustand und DOM divergieren	Deklarativ: UI = f(state)

Der konzeptionelle Unterschied

- **Imperativ** (Vanilla JS): „Finde Element X, ändere Property Y, füge Z ein“
- **Deklarativ** (React/Vue): „So soll die UI aussehen, wenn der Zustand S ist“ — Framework berechnet die DOM-Änderungen

Frameworks lösen nicht ein Syntax-Problem, sondern ein **Zustandsmanagement-Problem**: Wie bleibt die UI konsistent, wenn sich Daten ändern?

Die zentrale Erkenntnis: Frameworks wie React sind keine syntaktischen Verschönerungen, sondern eine **konzeptionelle Verschiebung**. Imperatives DOM („greife Element, ändere Property“) wird durch deklarative UI ersetzt („so soll es aussehen, wenn der Zustand X ist“). Das Framework berechnet automatisch die minimalen DOM-Änderungen — das sogenannte „Reconciliation“ oder „Virtual DOM Diffing“ bei React. Für kleine Anwendungen ist das Overkill, weil 50 Zeilen Vanilla JS reichen. Sobald aber Zustand an mehreren Stellen synchron gehalten werden muss (Warenkorb-Badge, Sidebar, Modal, Hauptanzeige), wird imperatives DOM-Tracking unwartbar und der Framework-Overhead lohnt sich.

JavaScript-Ökosystem: Überblick

Bereich	Technologie	Rolle
Browser-Frameworks	React, Vue.js, Angular, Svelte	Deklarative, komponentenbasierte UI
Server-Runtime	Node.js, Deno, Bun	Dasselbe Event-Loop-Modell serverseitig
Full-Stack Frameworks	Next.js, Nuxt.js, SvelteKit	Server-Rendering + Client-Interaktion
TypeScript	Typisierte Obermenge von JS	Statische Typprüfung für große Codebases
Build-Tools	Vite, webpack, esbuild	Bündelung, Transpilation, Optimierung

Die Wahl des Frameworks ist eine **Architekturentscheidung** — sie bestimmt, wie Zustand verwaltet, Code organisiert und die Anwendung deployt wird.

Diese Tabelle ist bewusst nicht vollständig — das JavaScript-Ökosystem ist riesig und ändert sich schnell. Wichtig sind die *Kategorien*: Browser-Frameworks (UI), Server-Runtimes (Node.js und Konkurrenz), Full-Stack-Frameworks (Server + Client kombiniert), TypeScript als Typsystem und Build-Tools für die Auslieferung. Wer diese fünf Kategorien kennt, kann jede neue Technologie sofort einordnen. Die Wahl ist nie technisch neutral: Sie bestimmt, wie das Team arbeitet, welche Bibliotheken zur Verfügung stehen, wie deployment funktioniert und wie schwer es ist, neue Entwickler einzuarbeiten.

Takeaway

JavaScript ist die **Verhaltensschicht** des Webs und zugleich der Orchestrator: Es ist die einzige Technologie, die sowohl DOM (HTML) als auch CSSOM (CSS) verändern kann. Das Ausführungsmodell — single-threaded mit Event Loop — ermöglicht responsive UIs trotz Netzwerk-Latenz. Die Fetch API verbindet Frontend mit Backend. Für Web Engineering sind drei Konzepte entscheidend: das **Event-Loop-Modell**, die **Abhängigkeitsrichtung** (JS erweitert HTML/CSS, nicht umgekehrt) und die **Grenze zu Frameworks** (ab welcher Komplexität lohnt deklarative UI).

Das Event-Loop-Modell ist das einheitliche Erklärungsmodell für scheinbar unzusammenhängende JavaScript-Verhaltensweisen: warum der Browser trotz single-threaded Ausführung nicht einfriert (asynchrone Callbacks warten in der Queue, nicht im Stack), warum `async/await` syntaktischer Zucker über Promises ist und keine echte Parallelität erzeugt, warum eine synchrone Schleife die UI blockiert (der Event Loop kann keine anderen Callbacks verarbeiten, solange der Stack besetzt ist), und warum Node.js tausende gleichzeitige Netzwerkverbindungen mit einem einzigen Thread bedienen kann (I/O ist nie blocking — die Callbacks warten in der Queue). DOM, Events, Fetch und Frameworks bauen alle auf diesem Modell auf.

CSS — Layout und Responsive Design

Leitfrage: Warum braucht das Web eine eigene Sprache für Darstellung — und wo endet die Verantwortung von CSS, wo beginnt die von JavaScript?

CSS ist die anspruchsvollste der drei Frontend-Sprachen, weil sie vollständig deklarativ ist: Die Entwicklerin beschreibt Constraints, der Browser löst sie auf — der konkrete Pixel-Wert ist nie direkt spezifizierbar. Die meisten schwer debugbaren Bugs in Frontend-Code sind keine JavaScript-Fehler, sondern unverständene CSS-Spezifität, Cascade oder Layout-Wechselwirkungen. Die Kaskade, das Spezifitätssystem und das Box-Modell sind keine Details, sondern die Kernmechanik — wer sie versteht, kann jedes CSS-Framework schnell lesen und anpassen, weil alle Frameworks auf denselben Browser-Grundlagen aufbauen.

Welche Farbe hat der Text?

```
<p id="info" class="highlight">Willkommen im Shop</p>
```

```
p      { color: blue; }  
.highlight { color: green; }  
#info   { color: red; }  
p.highlight { color: orange; }
```

Aufgabe: Der Paragraph matcht alle vier Selektoren. Welche Farbe wird angezeigt — und warum?

Die häufigste Fehlintuition bei CSS ist, dass die Reihenfolge im Stylesheet entscheidet — Spezifität hat aber Vorrang. Die Reihenfolge im Stylesheet ist nur der Tiebreaker, wenn zwei Regeln identische Spezifität haben. Das Spezifitätssystem ist deterministisch: $(1, 0, 0)$ (ID) schlägt immer $(0, 99, 99)$ (Klassen + Elemente). Diese Fehlintuition ist die Wurzel des häufigsten CSS-Bug-Musters — eine Regel wird hinzugefügt, greift aber nicht, weil eine existierende Regel mit höherer Spezifität gewinnt. Der DevTools-Inspector zeigt überschriebene Eigenschaften mit einem Durchstreichen — das ist das wichtigste CSS-Debug-Werkzeug im Browser.

Auflösung: Spezifität entscheidet

Selektor	Spezifität (ID, Klasse, Element)	Farbe
p	(0, 0, 1)	blue
.highlight	(0, 1, 0)	green
p.highlight	(0, 1, 1)	orange
#info	(1, 0, 0)	red

ID schlägt Klasse schlägt Element. In großen Projekten führen Spezifitätskonflikte zu unerwartetem Verhalten — deshalb setzen moderne Projekte auf Konventionen (BEM) oder komponentenbezogenes CSS (Scoped Styles).

Die Spezifität wird als Tripel (ID, Klasse, Element) berechnet und lexikografisch verglichen: $(1, 0, 0)$ schlägt $(0, 99, 99)$. Inline-Styles (`style="..."`) haben die noch höhere Spezifität $(1, 0, 0, 0)$. `!important` überschreibt alles — und ist deshalb verpönt, weil es die Kaskade außer Kraft setzt und spätere Überschreibungen unmöglich macht. BEM (Block-Element-Modifier) ist eine Namenskonvention, die Spezifitätskonflikte vermeidet, indem jede Komponente mit nur einer Klasse gestylt wird. Scoped Styles in Vue oder CSS Modules in React lösen dasselbe Problem automatisch, indem sie Klassennamen pro Komponente eindeutig machen.

Warum existiert CSS als eigene Technologie?

In den 1990ern wurde Darstellung direkt in HTML geschrieben: `<font color="red" size="5">`. Das führte zu einem fundamentalen Problem:

Problem	Ohne CSS	Mit CSS
Redesign	Jede HTML-Seite einzeln ändern	Ein Stylesheet ändern → alle Seiten betroffen
Verschiedene Geräte	Eine Version für alle	Verschiedene Stylesheets: Screen, Print, Mobile
Barrierefreiheit	Visuelles Layout = Dokumentstruktur	Screenreader ignoriert CSS, liest Struktur
Teamarbeit	Designer ändert HTML	Designer ändert CSS, Entwickler ändert HTML

Das Kernkonzept: Ein HTML, viele Darstellungen

Dieselbe Shop-Seite kann mit **verschiedenem CSS** als Desktop-Layout, als mobile Ansicht, als Druckversion oder im Dark Mode erscheinen — ohne das HTML zu ändern. Das ist kein Feature, sondern der **Grund, warum CSS als separate Sprache existiert**.

Normal Flow zuerst



- HTML-Elemente folgen zunächst dem **normal flow**
- **Block-Elemente** beginnen auf einer neuen Zeile und nehmen typischerweise die volle verfügbare Breite ein
- **Inline-Elemente** bleiben in derselben Zeile und nehmen nur so viel Platz ein wie ihr Inhalt braucht
- Erst danach greifen `display`, `position`, Flexbox oder Grid als bewusste Abweichungen vom Standard

So sieht das aus

```
<p>Absatz <span>Inline-Text</span> nach dem Wort.</p>  
<div>Dieses Block-Element steht in einer eigenen Zeile.</div>
```

Im Browser: Der Absatztext läuft in einer Zeile, das `span` bleibt im Textfluss, und das `div` startet darunter auf einer neuen Zeile.

CSS ist deklarativ, nicht imperativ

CSS sagt nicht „zeichne ein Rechteck bei Pixel 200,300“. CSS sagt „dieses Element soll 80% breit sein, zentriert, mit 1rem Abstand“. Der **Browser** berechnet die konkreten Pixel. Das ist fundamental anders als Canvas/JavaScript-Zeichnen und der Grund, warum CSS-Layouts automatisch auf verschiedene Bildschirmgrößen reagieren können.

Der historische Kontext ist wichtig: Vor CSS wurden Webseiten mit `<font>`, `<center>`, `<table>` für Layout und inline-Attributen wie `bgcolor` gestylt. Das hatte drei Probleme: Wartung (jede Seite einzeln ändern), Gerätevielfalt (eine Version für alle) und Barrierefreiheit (Layout-Tabellen verwirren Screenreader). CSS wurde 1996 spezifiziert, um diese Probleme zu lösen — indem Darstellung von Struktur getrennt wurde. Die Trennung ist so fundamental, dass sie bis heute das Grundprinzip des Webs ist — selbst Komponenten-Frameworks, die HTML/CSS/JS bündeln, halten die Schichten konzeptionell getrennt. Der Normal Flow ist die Default-Anordnung: block elements stapeln sich vertikal, inline elements fließen horizontal. Erst `display`, `position`, Flexbox oder Grid verändern das bewusst.

CSS in Kurzform

CSS besteht aus **Selektoren** und **Deklarationen**:

```
p.highlight {  
  color: orange;  
  font-weight: 600;  
}
```

- **Selektor:** `p.highlight` sagt, *welche* Elemente gemeint sind
- **Deklaration:** `color: orange;` sagt, *wie* sie aussehen sollen
- **Regelblock:** alles zwischen `{ ... }`

Was CSS auflöst

- **Kaskade:** mehrere Regeln können denselben Stil setzen
- **Spezifität:** manche Selektoren gewinnen bei Konflikten
- **Vererbung:** manche Eigenschaften wandern von Eltern zu Kindern

Merksatz: CSS beschreibt Stil als Regeln, nicht als Pixelbefehle.

Pseudo-Klassen



Pseudo-Klassen sind Selektoren für **Zustände** oder **Positionen** eines Elements. Sie beginnen mit `:` und hängen an einen normalen Selektor an.

```
a:hover {  
  text-decoration: underline;  
}  
  
input:focus {  
  border-color: #1a5fa8;  
}
```

- `:hover` trifft ein Element, wenn der Mauszeiger darüber liegt
- `:focus` trifft ein Element, wenn es per Tastatur oder Klick aktiv ist
- Pseudo-Klassen beschreiben also nicht *was* ein Element ist, sondern *in welchem Zustand* es gerade ist

Zwei typische Beispiele:

- `button:hover` für Hover-Effekte
- `input:focus` für fokussierte Formularfelder

Pseudo-Klassen sind der Grund, warum CSS *reaktiv* ohne JavaScript sein kann. `:hover` reagiert auf Maus-Position, `:focus` auf Tastatur-Navigation, `:checked` auf Checkbox-Zustand, `:valid/:invalid` auf Formular-Validität. Die Kaskade wird regelmäßig mit dem „Kaskadierungsprinzip“ erklärt: Von mehreren gültigen Regeln gewinnt die spezifischste, bei gleicher Spezifität die spätere. Diese Sortierung ist deterministisch — aber das Tripel aus Ursprung, Spezifität und Reihenfolge ist für Anfänger die häufigste Verwirrungsquelle. Die Vererbung ist ein zweites Konzept: manche Eigenschaften (`color`, `font`) wandern von Eltern zu Kindern, andere (`border`, `padding`) nicht.

Was CSS ohne JavaScript kann

Bevor man JavaScript für visuelle Effekte einsetzt, lohnt die Frage: Kann CSS das allein?

Anforderung	CSS-Lösung	JS nötig?
Hover-Effekt auf Produktkarte	:hover Pseudo-Klasse	nein
Warenkorb-Button-Animation	transition + :active	nein
Dark Mode	@media (prefers-color-scheme: dark)	nein
Formularfeld-Validierung visuell	:valid, :invalid, :focus	nein
Akkordeon (auf/zuklappen)	<details> + CSS	nein
Smooth Scroll	scroll-behavior: smooth	nein
Drag & Drop sortieren	—	ja
Daten vom Server nachladen	—	ja

Die Grenze

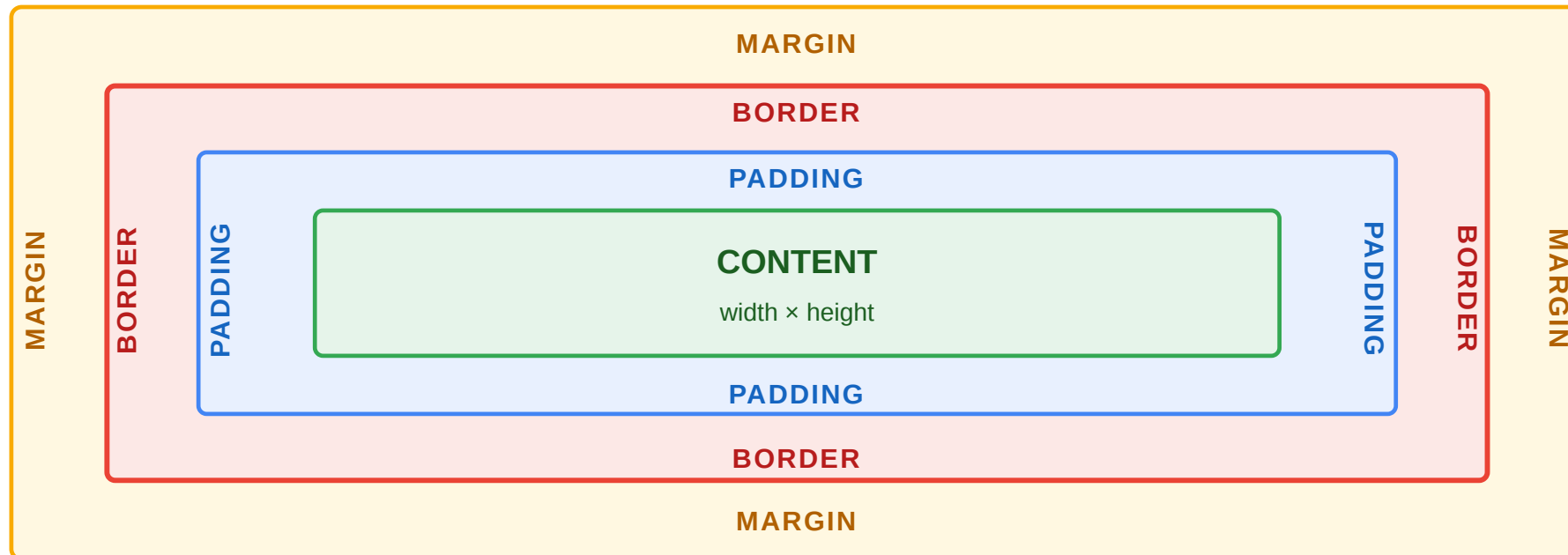
CSS reagiert auf **Zustände** (:hover, :checked, :focus, @media) und **animiert Übergänge**. Sobald **neuer Zustand erzeugt** werden muss (Daten laden, DOM-Elemente erstellen, Server kontaktieren), braucht man JavaScript. JavaScript sollte den **Auslöser** steuern (Klasse setzen), CSS die **Darstellung** (Transition/Animation).

CSS deckt seit 2020 mehr Interaktionsmuster ab als allgemein angenommen: Akkordeons mit dem nativen <details>/<summary>-Element, Dropdown-Menüs mit :focus-within, Formular-Validierung mit :valid/:invalid/:user-invalid, Themes mit Custom Properties und @media (prefers-color-scheme: dark), Smooth Scroll mit scroll-behavior: smooth. CSS-Lösungen sind wartungsärmer (kein State-Management), performanter (keine JavaScript-Evaluation im Render-Pfad) und barrierefreundlicher (Browser-natives Verhalten). JavaScript ist nur notwendig, wenn neue Zustände aus externen Quellen entstehen: Netzwerkdaten, berechnete Werte oder dynamisch erzeugte DOM-Strukturen.

CSS-Layouts: Das Box-Modell

HTML Elemente müssen nun positioniert werden. Dazu wird jedes HTML-Element als eine Box betrachtet: **Content** → **Padding** → **Border** → **Margin**

Die häufigste Fehlerquelle: `width: 200px` bedeutet standardmäßig nur den Content — Padding und Border kommen **obendrauf**. Erst `box-sizing: border-box` macht `width` zum Gesamtmaß.



Das Box-Modell ist das erste wirklich nicht-intuitive Detail von CSS: `width: 200px + padding: 20px + border: 2px` ergibt im Default-Box-Modell ein Element mit 244px tatsächlicher Breite — und zerschießt jedes Raster-Layout. Deshalb beginnt fast jedes moderne Stylesheet mit `* { box-sizing: border-box; }` als Reset. Dann entspricht `width: 200px` der tatsächlichen Breite, und Padding und Border werden innen abgezogen. Das ist eine der wenigen Stellen, an denen CSS einen schlechten Default hat — historisch gewachsen und nicht mehr änderbar, weil es unzählige bestehende Webseiten brechen würde.

Layoutmodelle im Überblick

Nicht jedes CSS-Layoutproblem ist dasselbe. Die wichtigsten Modelle beantworten unterschiedliche Fragen:

Normal Flow

Default-Verhalten von HTML:

- Block-Elemente stapeln sich untereinander
- Inline-Elemente fließen im Text
- Ausgangspunkt für jedes Layout

Grid

Organisiert Elemente in **Zeilen und Spalten**:

- zweidimensional
- explizite Rasterstruktur
- ideal für Seitenlayout, Dashboards, Galerien

Flexbox

Verteilt Elemente entlang **einer Achse**:

- horizontal oder vertikal
- mit Abstand, Reihenfolge und Ausrichtung
- ideal für Reihen, Menüs, Toolbars

Faustregel

- **Flow**: Was passiert ohne Layoutanweisung?
- **Flexbox**: Wie verteilen wir Inhalt entlang einer Achse?
- **Grid**: Wie definieren wir ein Raster?

Die didaktisch wichtigste Unterscheidung ist: Flow, Flexbox und Grid sind keine drei Varianten desselben Werkzeugs, sondern drei verschiedene Abstraktionen. Normal Flow ist der Ausgangszustand des Dokuments. Flexbox löst Verteilungsprobleme entlang einer Dimension, etwa „wie stehen diese Navigationspunkte nebeneinander und wie verteilen sie freien Platz?“. Grid löst Strukturprobleme in zwei Dimensionen, etwa „welche Bereiche gibt es im Layout und wie liegen sie in Zeilen und Spalten?“. Viele CSS-Probleme werden unnötig kompliziert, weil man mit dem falschen Modell startet.

Normal Flow: Das Default-Layout des Browsers

Bevor wir Flexbox oder Grid einsetzen, sollten wir verstehen, was HTML **ohne besondere Layoutregeln** tut:

Block-Elemente beginnen typischerweise in einer neuen Zeile und nehmen die verfügbare Breite ein.

```
<p>Absatz <span>Inline-Text</span> nach dem Wort.</p>  
<div>Dieses Block-Element steht darunter.</div>
```

Inline-Elemente bleiben im laufenden Text und belegen nur so viel Platz wie ihr Inhalt braucht.

Absatz **Inline-Text** nach dem Wort.

Dieses Block-Element steht darunter.

Flow ist deshalb wichtig, weil Flexbox und Grid immer bewusst vom Normalfall abweichen.

Normal Flow ist kein primitives Altverhalten, sondern die Basis des Webs. Dokumente, Artikel, Listen, Formulare und Absätze funktionieren bereits sinnvoll, bevor irgendein Layoutsystem aktiviert wird. Genau deshalb sollte man mit semantischem HTML beginnen und Layout erst danach ergänzen. Historisch waren viele CSS-Probleme eigentlich Missverständnisse des Normal Flow: Wer nicht versteht, warum ein `div` unter einem `p` steht oder warum Inline-Elemente keine eigene Blockhöhe erzeugen, wird auch Flexbox und Grid nur mechanisch anwenden.

Flexbox: Verteilung entlang einer Achse

Flexbox ist das richtige Werkzeug, wenn Elemente in **einer Zeile oder Spalte** organisiert werden sollen:

```
.nav {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  gap: 1rem;  
}
```

Logo

Produkte

Warenkorb

Typische Fragen:

- nebeneinander oder untereinander?
- wie viel Abstand?
- links, mittig oder rechts ausrichten?

Flexbox ist ideal für:

- Navigation
- Button-Gruppen
- Kartenreihe
- Toolbar oder Header-Leiste

Flexbox ist ein eindimensionales Layoutmodell. Die Hauptachse wird durch `flex-direction` festgelegt, die Querachse ergibt sich daraus. `justify-content` arbeitet auf der Hauptachse, `align-items` auf der Querachse. Genau diese Achsenlogik macht Flexbox für Menüs, Toolbars und Kartenreihen so stark. Es ist aber das falsche Modell, wenn die Beziehung zwischen Zeilen und Spalten explizit beschrieben werden soll. Dann zwingt man Flexbox in Probleme hinein, die Grid klarer löst.

Grid: Ein explizites 2D-Raster

Grid ist sinnvoll, wenn das **Layout selbst** die Struktur vorgibt.

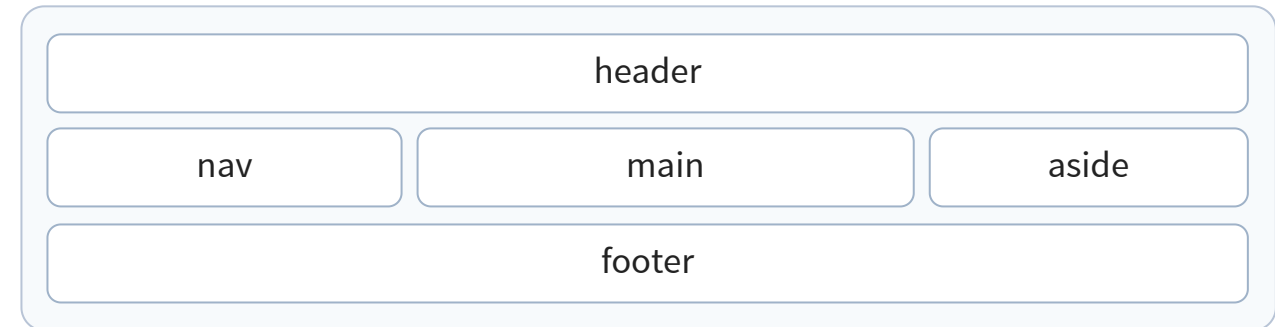
Explizit bedeutet hier: In CSS werden **Spalten**, **Zeilen** und oft auch die **Platzierung der Bereiche** vorgegeben.

```
.shop-layout {  
  display: grid;  
  grid-template-columns: 240px 1fr 220px;  
  grid-template-rows: auto 1fr auto;  
  grid-template-areas:  
    "header header header"  
    "nav   main   aside"  
    "footer footer footer";  
  gap: 1rem;  
}
```

1fr bedeutet: nimm **einen Anteil des verbleibenden freien Platzes**.

"header header header" bedeutet: Der Bereich header belegt in dieser Zeile **alle drei Spalten**.

grid-template-columns definiert die **Spalten**.
grid-template-rows definiert die **Zeilen**.
grid-template-areas ordnet benannte Bereiche im Raster an.



Bei Grid entsteht zuerst das Raster. Danach werden die Elemente in dieses Raster eingeordnet.

„Explizites 2D-Raster“ sollte hier technisch verstanden werden: Die Autorin beschreibt in CSS zunächst die **Tracks** des Grids, also Spalten und Zeilen. Erst danach werden Elemente auf diese Struktur abgebildet, entweder automatisch oder explizit über `grid-area`, `grid-column` und `grid-row`. Das unterscheidet Grid von Flexbox. Bei Flexbox entsteht die Anordnung aus der Reihenfolge der Items entlang einer Achse; bei Grid wird zuerst die räumliche Struktur definiert und dann belegt. `1fr` ist die zentrale Maßeinheit in Grid und bedeutet „ein Anteil des noch freien Platzes“. In `240px 1fr 220px` sind die äußeren Spalten fest, die mittlere Spalte bekommt den restlichen Raum. Die Zeichenkette `"header header header"` ist keine Wiederholung ohne Bedeutung, sondern markiert drei Zellen derselben Zeile mit demselben Namen. Dadurch spannt sich der Bereich `header` über alle drei Spalten.

Responsives Grid ohne starre Breakpoints

Ein Grid kann responsiv werden, ohne für jedes Gerät ein eigenes Layout zu definieren:

```
.product-grid {  
  display: grid;  
  gap: 1rem;  
  grid-template-columns:  
    repeat(auto-fit, minmax(220px, 1fr));  
}
```

`minmax(220px, 1fr)` bedeutet:

- nie schmaler als 220px
- sonst flexibel wachsen

`auto-fit` versucht so viele Spalten wie möglich in die Zeile zu legen.

Produkt A

Produkt B

Produkt C

⋮

Damit entsteht ein fluides Raster:

- auf schmalen Screens wenige Spalten
- auf breiten Screens mehr Spalten
- ohne duplizierte Layouts

`auto-fit` und `minmax()` sind das Kernmuster moderner responsiver Kartenraster. Der Browser berechnet selbst, wie viele Spalten bei gegebener Breite sinnvoll sind. Das ist ein gutes Beispiel für die deklarative Stärke von CSS: Man beschreibt Grenzen und Flexibilität, nicht einzelne Pixelkoordinaten. Der Unterschied zu klassischen Breakpoint-Layouts ist didaktisch wichtig: Statt „bei 768px zwei Spalten, bei 1200px vier Spalten“ beschreibt man eine allgemeine Regel, und der Browser löst sie kontinuierlich auf.

Beispiel: Produktdetailseite mit Bild und Text

Situation: Links steht das Produktbild, rechts Beschreibung und Kaufbutton. Auf dem Smartphone soll das Bild oben und der Text darunter erscheinen.

Mögliche Wahl: Flexbox

- funktioniert für einfache Zeile/Spalte
- gut bei linearen Anordnungen
- weniger explizit bei komplexer Umordnung

Mögliche Wahl: Grid

- beschreibt Desktop als "img text"
- beschreibt Mobile als "img" / "text"
- Layout bleibt als Struktur sichtbar

Dieses Beispiel ist wichtig, weil es zeigt, dass die Wahl nicht dogmatisch sein darf. Flexbox ist nicht falsch, wenn Bild und Text einfach nebeneinander oder untereinander stehen. Grid wird stärker, sobald das Layout selbst explizit modelliert werden soll, etwa mit benannten Bereichen, stabilen Relationen oder mehreren Umbrüchen. Die fachliche Lektion lautet deshalb nicht „Grid ist moderner“, sondern „wähle das Modell, das die räumliche Struktur am direktesten ausdrückt“.

Welche Wahl ist hier besser?

Flexbox passt, wenn

- nur eine Hauptachse wichtig ist
- die Elemente einfach verteilt werden
- Reihenfolge und Struktur weitgehend linear bleiben

Grid passt, wenn

- Zeilen und Spalten gemeinsam wichtig sind
- Bereiche explizit benannt werden sollen
- responsives Umschalten strukturell klar bleiben soll

Faustregel: Flexbox, wenn der Inhalt verteilt wird. Grid, wenn das Layout als Raster vorgegeben wird.

In der Praxis arbeiten beide Modelle zusammen. Grid organisiert häufig die grobe Seitenstruktur, Flexbox den Inhalt einzelner Bereiche wie Header, Kartenkopf oder Button-Leiste. Das ist kein Widerspruch, sondern die normale Architektur moderner Oberflächen. Wer versucht, alles nur mit Flexbox oder alles nur mit Grid zu lösen, verliert Ausdruckstärke und Lesbarkeit. Gute CSS-Architektur beginnt daher mit der Frage: Welche räumliche Beziehung will ich eigentlich ausdrücken?

Takeaway

CSS existiert als eigene Sprache, weil dasselbe HTML in **verschiedenen Kontexten** verschieden dargestellt werden muss — Screen, Print, Mobile, Dark Mode. CSS ist **deklarativ**: Man beschreibt Constraints, der Browser berechnet Pixel. Spezifität bestimmt, welche Regel gewinnt. Flexbox und Grid lösen unterschiedliche Verteilungsprobleme (Inhalt verteilen vs. Raster vorgeben). Responsive Design mit Mobile-First ist Voraussetzung. Die wichtigste Grenzfrage: **Kann CSS das allein** (Hover, Transition, Media Query), oder braucht es JavaScript? Nur wenn neuer Zustand erzeugt werden muss, ist JS die richtige Schicht.

Deklarativität ist das definierende Merkmal von CSS: Der Entwickler beschreibt Constraints (Breite, Abstände, Farbe, Layout-Regeln), der Browser berechnet die konkreten Pixel — abhängig von Viewport, Schriftgröße, Gerätepixeldichte und verfügbarem Raum. Das ist fundamental anders als Canvas-Rendering oder SVG-Zeichnen mit Pixelkoordinaten. Die Deklarativität ist der Grund, warum `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr))` automatisch auf 300px-Screens mit einer Spalte und auf 1400px-Screens mit sechs Spalten reagiert — ohne Media Query. Imperative CSS-Ansätze (z.B. Breiten per JavaScript explizit setzen) kämpfen gegen dieses Modell und erzeugen Layout-Bugs bei Reflows und Resize-Events.

Web APIs — die Schnittstellen des Browsers

Leitfrage: Was sind Web APIs — und warum gehören sie nicht zur JavaScript-Sprache selbst, sondern zum Browser?

Die JavaScript-Sprache (ECMAScript) ist erstaunlich klein: Variablen, Funktionen, Schleifen, Objekte — keine Datei-, Netzwerk- oder DOM-Operationen. Alles, was JavaScript im Browser *praktisch* tun kann, kommt von **Web APIs** — Schnittstellen, die der Browser bereitstellt. Das ist wichtig für die Architektur, weil derselbe JavaScript-Code in unterschiedlichen Umgebungen unterschiedliche APIs hat: im Browser `window` und `fetch`, in Node.js `process` und `fs`, in einem Service Worker weder noch das eine noch das andere.

Web APIs in einem Satz

Web APIs sind Browser-Schnittstellen, die JavaScript zusätzliche Fähigkeiten geben: den DOM verändern, Netzwerke ansprechen, Daten speichern, Gerätefunktionen nutzen oder Medien steuern.

Typische Browser-APIs

- DOM API
- Fetch API
- Web Storage API
- Geolocation API
- Canvas / WebGL
- Web Workers

Warum das wichtig ist

- JavaScript allein beschreibt die Sprache
- Web APIs machen sie im Browser praktisch nutzbar
- viele Web-Funktionen sind erst durch diese APIs möglich

Beispiel: `fetch()` lädt Daten, `localStorage` speichert Zustand, DOM APIs aktualisieren die Oberfläche.

Der Begriff „API“ ist im Web-Kontext doppeldeutig: Er kann eine **Web API** im Sinne einer Browser-Schnittstelle meinen (was dieses Modul behandelt) oder eine **HTTP-API** im Sinne eines serverseitigen Endpunkts (was Vorlesung 04 behandelt). Beide sind APIs im breiten Sinn „Application Programming Interface“. Wenn Sie in diesem Modul von APIs sprechen, meinen Sie Schnittstellen *des Browsers* an den JavaScript-Code. Diese kommen nicht aus ECMAScript selbst — JavaScript als Sprache kennt weder `window`, noch `document`, noch `fetch`. Es sind Erweiterungen, die der Browser zur Laufzeit bereitstellt.

Wofür nutzt man sie?

Aufgabe	Beispiel-API
HTML oder CSS zur Laufzeit ändern	DOM API
Daten vom Server laden	Fetch API
Zustand im Browser speichern	Web Storage API
Standorte abfragen	Geolocation API
Lange Berechnungen auslagern	Web Workers

Kernaussage: Web APIs sind die Schnittstelle zwischen JavaScript und den Fähigkeiten des Browsers.

Grenze: Nicht jede API ist Teil von JavaScript selbst. Viele Funktionen kommen vom Browser, nicht von der Sprache.

Die Unterscheidung ist praktisch relevant: Derselbe JavaScript-Code läuft je nach Umgebung unterschiedlich. Im Browser funktioniert `localStorage`, in Node.js nicht. In Node.js funktioniert `fs.readFile`, im Browser nicht. Wer Isomorphic JavaScript (Code, der auf Server und Client läuft) schreibt, muss diese Unterschiede kennen — und muss manchmal mit Polyfills oder Feature-Detection (`if (typeof window !== 'undefined')`) arbeiten. Moderne Full-Stack-Frameworks wie Next.js abstrahieren diese Trennung teilweise, aber unter der Haube bleibt sie bestehen.

Fetch API im Detail: Request, Response, AbortController

```
const controller = new AbortController();

// Request mit Timeout nach 5 Sekunden abbrechen
const timeout = setTimeout(() => controller.abort(), 5000);

try {
  const response = await fetch('/api/products', {
    method: 'GET',
    headers: { 'Accept': 'application/json' },
    signal: controller.signal,
  });

  if (!response.ok) throw new Error(`HTTP ${response.status}`);

  const products = await response.json();
} catch (err) {
  if (err.name === 'AbortError') console.log('Request abgebrochen');
  else console.error('Netzwerkfehler', err);
} finally {
  clearTimeout(timeout);
}
```

Konzept	Zweck
Request / Response	Objektmodell für HTTP-Nachrichten
headers	Metadaten: Content-Type, Accept, Authorization
AbortController	Laufende Requests abbrechen (Timeout, Navigation)
response.ok	true bei Status 200–299

Der `AbortController` ist eine unterschätzte API. Ohne ihn läuft ein `Fetch-Request` weiter, auch wenn der Nutzer inzwischen weggenavigiert ist oder die Komponente unmounted wurde — der `Response` trifft dann auf DOM-Elemente, die nicht mehr existieren, und löst kryptische Fehler aus. Moderne Frameworks nutzen `AbortController` intern, um Cleanup zu machen. Für eigene `Fetch-Logik` ist er Pflicht, sobald es um Such-Eingaben, Navigation oder Timeouts geht. Das Pattern „Request starten, Abort-Signal merken, beim Abbruch `signal.abort()` aufrufen“ ist ein Muster, das Sie in jedem produktiven Frontend-Code sehen werden.

Web Storage: localStorage, sessionStorage, Cookies

API	Lebensdauer	Größe	Zugriff
localStorage	Persistent (bis manuell gelöscht)	~5–10 MB	Nur Client (JS)
sessionStorage	Nur aktuelle Browser-Session	~5–10 MB	Nur Client (JS)
Cookies	Konfigurierbar (Session oder Ablaufdatum)	~4 KB	Client + automatisch an Server
IndexedDB	Persistent	Gigabyte-Bereich	Client (asynchron)

```
// localStorage: einfache Key-Value-Speicherung
localStorage.setItem('cart', JSON.stringify(cartItems));
const cart = JSON.parse(localStorage.getItem('cart') || '[]');

// sessionStorage: nur für die aktuelle Tab-Session
sessionStorage.setItem('wizard-step', '3');

// Cookies: werden bei jedem HTTP-Request mitgesendet
document.cookie = 'theme=dark; max-age=31536000; SameSite=Strict';
```

Wann was?

- **Cookies** für alles, was der **Server wissen muss** (Session-ID, Authentifizierung)
- **localStorage** für Client-State, der **über Sessions** erhalten bleiben soll (Warenkorb, UI-Einstellungen)
- **sessionStorage** für temporären State, der **nur im aktuellen Tab** gilt (Wizard-Fortschritt)
- **IndexedDB** für große Datenmengen oder Offline-Anwendungen

Ein häufiger Fehler: Authentifizierungs-Tokens in `localStorage` zu speichern. Das ist verlockend, weil es einfach ist, aber gefährlich: Jeder JavaScript-Code auf der Seite (inklusive eingebettete Drittanbieter-Skripte und XSS-Angriffe) kann `localStorage` lesen. Cookies mit `HttpOnly` und `SameSite=Strict` sind sicherer, weil sie JavaScript verborgen bleiben und nur vom Server gelesen werden können. Das ist eine der Architektur-Entscheidungen, die Vorlesung 7 (Sicherheit) vertieft. Für Warenkorb und UI-State ist `localStorage` dagegen völlig in Ordnung — kein Angreifer gewinnt etwas durch Kenntnis des Warenkorbinhalts.

History API: Navigation ohne Seiten-Reload

Single-Page Applications navigieren ohne vollständigen Seiten-Reload. Der Browser-Zurück-Button muss trotzdem funktionieren. Die **History API** macht das möglich:

```
// URL ändern, ohne neu zu laden
history.pushState({ page: 'products' }, '', '/products');

// Neue Inhalte fetch'en und rendern
const data = await fetch('/api/products').then(r => r.json());
renderProducts(data);

// Zurück-Button abfangen
window.addEventListener('popstate', (event) => {
  if (event.state?.page === 'products') renderProducts(/* ... */);
});
```

Methode / Event	Zweck
history.pushState(state, '', url)	URL ändern, Eintrag zum Verlauf hinzufügen
history.replaceState(...)	URL ändern, ohne neuen Eintrag
popstate Event	Browser-Zurück-/Vorwärts-Button

Die History API ist der technische Kern jedes Client-Side-Routers — React Router, Vue Router und SvelteKit nutzen sie unter der Haube. Das Problem, das sie löst: Ohne History API würde jede Navigation in einer SPA die URL unverändert lassen, und der Browser-Zurück-Button würde die SPA verlassen. Mit `pushState` bleibt die URL konsistent mit dem UI-Zustand, und der Zurück-Button funktioniert wie erwartet. Wichtige Feinheit: `pushState` löst *kein* `popstate`-Event aus — das Event feuert nur beim Zurück-Navigieren. Eigene Navigationslogik muss also zwei Pfade unterstützen: aktives Navigieren (UI ruft `pushState` + Render auf) und passives Zurück-Navigieren (`popstate`-Event triggert Render).

Web Workers vs Service Workers

Beide laufen in einem **separaten Thread** außerhalb des Haupt-Event-Loops — aber mit unterschiedlichen Zwecken.

Aspekt	Web Worker	Service Worker
Zweck	CPU-intensive Berechnungen auslagern	Netzwerk-Proxy, Offline-Support, Push
Lebensdauer	Gebunden an Tab/Seite	Persistent, auch wenn Tab geschlossen
DOM-Zugriff	Nein	Nein
Fetch-Interception	Nein	Ja — fängt alle Netzwerk-Requests ab
Typischer Einsatz	Bildverarbeitung, Datenanalyse, Kryptographie	Progressive Web Apps, Caching, Push-Benachrichtigungen

```
// Web Worker: teure Berechnung auslagern
const worker = new Worker('compute.js');
worker.postMessage({ data: largeDataset });
worker.onmessage = (e) => displayResult(e.data);

// Service Worker: Fetch abfangen für Offline-Support
self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request).then(cached => cached || fetch(event.request))
  );
});
```



Service Workers sind das Fundament jeder Progressive Web App (PWA). Sie ermöglichen Offline-Betrieb, weil sie Netzwerk-Requests abfangen und aus einem Cache beantworten können — selbst wenn die Verbindung weg ist. Web Workers dagegen lösen ein ganz anderes Problem: Lange Berechnungen (Bildverarbeitung, Parsen großer JSON-Dateien, Kryptographie) blockieren sonst die UI. Im Worker läuft dieser Code in einem echten zweiten OS-Thread, ohne den Hauptthread zu beeinträchtigen. Beide haben *keinen* DOM-Zugriff — sie kommunizieren mit dem Hauptthread über `postMessage`. Das ist wichtig, weil es bedeutet, dass Worker keine UI direkt ändern können; sie liefern nur Daten zurück.

Takeaway

Web APIs sind die **Fähigkeiten**, die der Browser JavaScript zur Verfügung stellt — nicht Teil der Sprache selbst, sondern Erweiterungen. Die wichtigsten Kategorien sind DOM-Manipulation, Netzwerk (Fetch), Speicher (localStorage/Cookies), Hintergrund-Threads (Web/Service Workers) und History. Welche API wofür eingesetzt wird, ist eine Architekturfrage mit Konsequenzen für Sicherheit, Performance und Offline-Fähigkeit.

Die Trennung zwischen ECMAScript (Sprache) und Web APIs (Plattform) hat konkrete technische Konsequenzen: Derselbe JavaScript-Code läuft in unterschiedlichen Umgebungen unterschiedlich — `localStorage` ist im Browser global verfügbar, in Node.js nicht definiert; `fs.readFile` ist in Node.js verfügbar, im Browser nicht. Polyfills implementieren eine Web API für Umgebungen, die sie nicht nativ unterstützen — z.B. `fetch` in älteren Node-Versionen. Feature-Detection (`if (typeof serviceWorker !== 'undefined')`) ist stabiler als Browser-Detection, weil sie direkt prüft ob die Fähigkeit vorhanden ist, unabhängig von Browser-Version oder User-Agent-String. Isomorphic JavaScript (Code, der auf Server und Client gleich läuft) muss diese Grenze mit Adapter-Schichten oder Conditional Imports explizit handhaben.

Das Zusammenspiel im Browser

Leitfrage: Wie arbeiten HTML, CSS und JavaScript im Browser zusammen — und warum ist das Verständnis dieser Pipeline für Performance und Architektur entscheidend?

Die gemeinsame Rendering-Pipeline des Browsers besteht aus sechs sequenziellen Phasen: (1) HTML parsen → DOM aufbauen, (2) CSS parsen → CSSOM aufbauen, (3) DOM + CSSOM → Render Tree (unsichtbare Elemente werden ausgeblendet), (4) Layout — Geometrie jedes Render-Tree-Knotens berechnen, (5) Paint — Pixel zeichnen, (6) Composite — Layer zusammenstellen. JavaScript kann in jeder Phase eingreifen: DOM-Mutationen invalidieren den Render Tree ab Schritt 3, `element.offsetHeight` erzwingt Layout synchron, CSS-Property-Änderungen mit `transform` können Schritte 4–5 überspringen. Performance-Probleme entstehen fast immer an diesen Schnittstellen, nicht innerhalb einer einzelnen Technologie.

Was passiert, wenn Sie „In den Warenkorb“ klicken?

Aufgabe: Skizzieren Sie den Ablauf: Welche Technologie (HTML, CSS, JavaScript) ist in welchem Schritt beteiligt? Was passiert sofort, was asynchron? Was sieht der Nutzer wann?

Der Warenkorb-Klick-Szenario veranschaulicht das Zusammenspiel aller drei Frontend-Technologien: HTML definiert Button und Badge, JavaScript registriert den Event-Handler und sendet den Fetch-Request, CSS rendert das visuelle Feedback (Button deaktiviert, Badge-Zähler). Die sechs Schritte auf der Folgefolie illustrieren das optimistische UI-Update-Muster: Schritte 2–3 (DOM-Änderung und CSS-Render) laufen synchron vor dem Netzwerk-Request, weil die wahrgenommene Latenz unter 100ms bleiben muss. Der asynchrone Fetch-Request (Schritt 4) kann in der Praxis 100–500ms dauern — nach dem sofortigen visuellen Feedback für den Nutzer nicht wahrnehmbar.

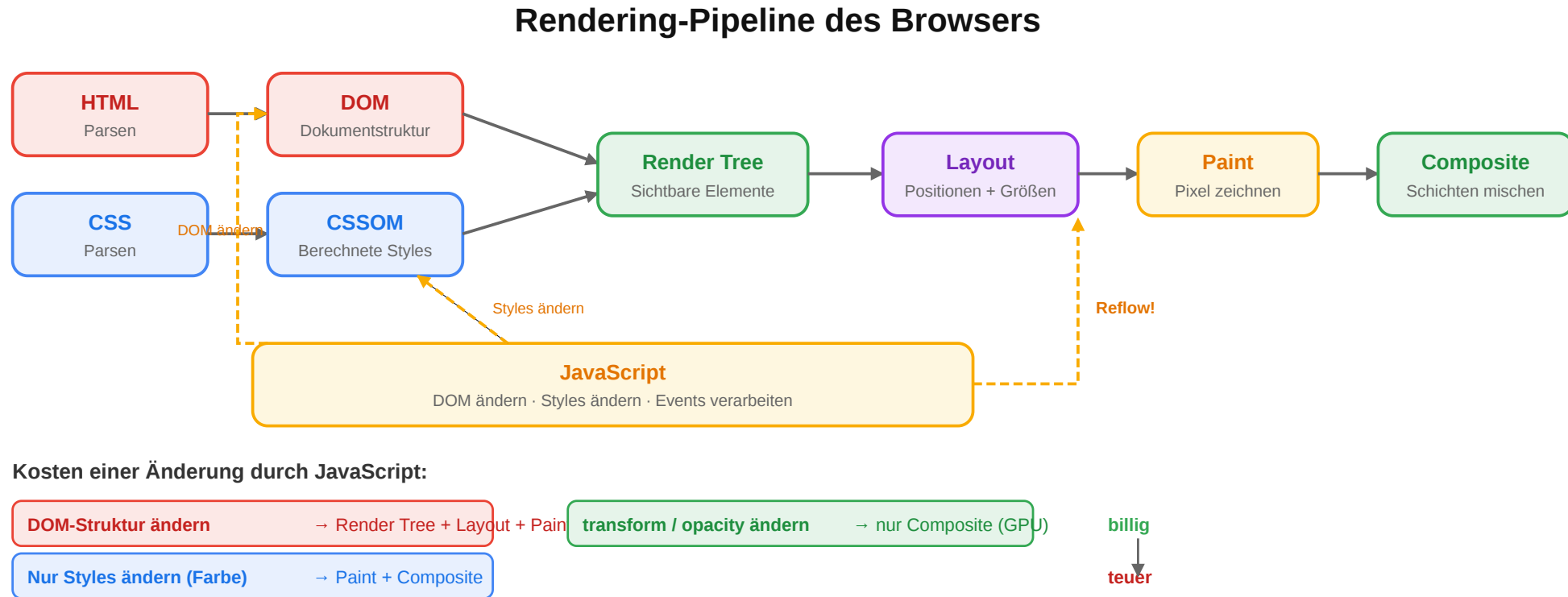
Ablauf (Skizze)

1. Nutzer klickt den Button → JavaScript reagiert auf das Event.
2. JavaScript aktualisiert DOM-Zustände (Badge, Button-Text).
3. CSS/Layout berechnet neues Rendering, der Nutzer sieht sofort Feedback.
4. JavaScript sendet asynchron POST `/api/cart` an den Server.
5. Server antwortet, JavaScript aktualisiert DOM erneut.
6. Browser führt Layout und Paint erneut aus.

Schlüsselbeobachtung: Das UI reagiert **sofort** (optimistisches Update), der Server wird **asynchron** kontaktiert. Das ist nur möglich, weil JavaScript non-blocking arbeitet.

Dieser Ablauf zeigt das Zusammenspiel: HTML liefert die Struktur (Button, Badge), JavaScript das Verhalten (Event-Handler, Fetch), CSS die Darstellung (Zustandsübergang, Animation). Die Aufteilung in sofortige und asynchrone Schritte basiert auf der 100ms-Wahrnehmungsschwelle (aus der HCI-Forschung von Nielsen/Miller): Reaktionen unter 100ms werden als direkte Kausalität wahrgenommen, darüber als Systemlatenz. Das optimistische UI-Update (Schritte 2–3 vor Schritt 4) hält das visuelle Feedback unter dieser Schwelle, selbst wenn der Server 300ms antwortet. Die Alternative — auf den Server warten, bevor die UI reagiert — wäre für Nutzer auf mobilen Netzen deutlich spürbar.

Die Rendering-Pipeline des Browsers



Die Pipeline arbeitet auf zwei internen Strukturen: dem **DOM** für HTML und dem **CSSOM** für CSS. Erst daraus erzeugt der Browser den Render Tree und rechnet Layout, Paint und Compositing aus.

Die Rendering-Pipeline ist deterministisch und läuft in sechs Phasen: (1) HTML parsen → DOM, (2) CSS parsen → CSSOM, (3) Render Tree aufbauen (DOM + CSSOM zusammenführen, dabei unsichtbare Elemente ausfiltern), (4) Layout (Berechnung von Position und Größe jedes Elements), (5) Paint (Pixel zeichnen), (6) Composite (Layer zusammenstellen). Bei jeder Änderung am DOM oder CSS fragt der Browser: Wie weit muss ich zurück in die Pipeline? Nur Farbe geändert? → nur Paint + Composite. Position geändert? → Layout + Paint + Composite (Reflow). Nur Transform oder Opacity? → nur Composite (am billigsten). Diese Unterscheidung ist der Schlüssel zu 60-FPS-Animationen.

Performance: Die häufigsten Fehler

Problem	Ursache	Lösung
Render-Blocking CSS	Browser wartet auf gesamtes CSS, bevor er etwas anzeigt	Critical CSS inline, Rest asynchron laden
Parser-Blocking JS	<script> stoppt HTML-Parsing	defer oder async Attribut
Layout Thrashing	JavaScript liest und schreibt abwechselnd DOM-Properties	Reads bündeln, dann Writes bündeln
Große JS-Bundles	Zu viel JavaScript auf einmal geladen	Code Splitting, Lazy Loading
Unoptimierte Bilder	5 MB Produktbilder ohne Kompression	WebP/AVIF, responsive srcset, Lazy Loading

Kernprinzip

Performance-Probleme im Web entstehen fast immer an der **Schnittstelle** von HTML, CSS und JavaScript — nicht innerhalb einer einzelnen Technologie. Deshalb muss man die Pipeline verstehen.

Layout Thrashing ist die am häufigsten übersehene Falle: Wenn JavaScript abwechselnd eine DOM-Property liest (`element.offsetHeight`) und eine schreibt (`element.style.height = ...`), muss der Browser *bei jedem Lesen* Layout neu berechnen, weil er nicht weiß, ob die vorherige Schreiboperation ihn invalidiert hat. Das Ergebnis: Aus einer 10-ms-Schleife wird eine 500-ms-Schleife. Lösung: Alle Reads zuerst, dann alle Writes. Die `requestAnimationFrame`-API ist der richtige Ort für DOM-Updates, weil der Browser dort weiß, dass danach ein Frame gerendert wird und er Batching betreiben kann.

Core Web Vitals: Wie Google Performance misst

Google misst Performance anhand dreier Nutzer-zentrierter Metriken, die auch ins Suchranking einfließen:

Metrik	Was es misst	Ziel
LCP — Largest Contentful Paint	Zeit, bis das größte sichtbare Element gerendert ist	< 2,5 s
INP — Interaction to Next Paint	Verzögerung zwischen Klick/Tap und sichtbarer Reaktion	< 200 ms
CLS — Cumulative Layout Shift	Wie stark sich das Layout nach Laden verschiebt	< 0,1

Typische Verursacher

- **Schlechter LCP:** große, unoptimierte Hero-Bilder; render-blocking CSS; langsamer Server
- **Schlechter INP:** lange Haupt-Thread-Blockaden durch JavaScript
- **Schlechter CLS:** Bilder ohne width/height; nachgeladene Werbung oder Fonts verschieben Inhalt

Warum das architektonisch wichtig ist

Core Web Vitals sind keine Laborwerte — sie werden am **echten Nutzer** gemessen (Real User Monitoring über Chrome UX Report). Eine Seite kann im Labor schnell wirken und in der Praxis trotzdem schlechte Werte haben, wenn sie auf langsamen Geräten oder Netzwerken läuft.

Core Web Vitals sind seit 2021 ein Ranking-Faktor bei Google — schlechte Werte können also direkte wirtschaftliche Konsequenzen haben. Die drei Metriken sind bewusst so gewählt, dass sie die drei wichtigsten Nutzerwahrnehmungen abbilden: „Ist die Seite schon da?“ (LCP), „Reagiert sie, wenn ich klicke?“ (INP), „Springt der Inhalt unter meinen Fingern weg?“ (CLS). Alle drei hängen direkt mit der Rendering-Pipeline zusammen: LCP ist ein Pipeline-Durchlauf, INP ist Hauptthread-Verfügbarkeit, CLS ist Reflow nach initialem Layout. Wer die Pipeline verstanden hat, weiß automatisch, wie man diese Metriken verbessert.

Wie hat sich das Zusammenspiel verändert?

Ära	Muster	Zusammenspiel
Klassisch (2000er)	Server rendert HTML, CSS als separate Datei, JS für kleine Effekte	Strikte Trennung: HTML-Datei + CSS-Datei + JS-Datei
jQuery-Ära (2006–2015)	HTML vom Server, JS manipuliert DOM direkt	DOM als zentrale Schnittstelle
Komponentenbasiert (ab 2013)	React/Vue/Angular: HTML + CSS + JS pro Komponente	Trennung nach Verantwortung , nicht nach Technologie
Server-First (ab 2020)	Next.js/SvelteKit: Server rendert, Client hydriert	Erst HTML vom Server, dann JavaScript macht es interaktiv

Was bedeutet „Client hydriert“?

Der Server liefert bereits fertiges HTML an den Browser. Danach lädt JavaScript nach und **bindet Verhalten an dieses vorhandene HTML**:

- Event-Listener werden registriert
- Komponenten bekommen ihren interaktiven Zustand
- die Seite wird nutzbar, ohne dass der Browser alles neu rendern muss

Kurz: Hydration bedeutet, dass der Client das serverseitig gerenderte HTML „zum Leben erweckt“.

Diskussionsfrage

Die klassische Web-Lehre sagt: „Trenne HTML, CSS und JavaScript in separate Dateien." React bündelt alle drei **pro Komponente**. Ist das ein Widerspruch zum Prinzip der Separation of Concerns — oder eine bessere Umsetzung davon?

Die vier Ären des Web-Frontends repräsentieren unterschiedliche Antworten auf dieselbe architektonische Frage: Wo gehört die Rendering-Logik hin? In den 2000ern blieb sie auf dem Server (HTML-Ausgabe). In der jQuery-Ära wanderte sie schrittweise in den Client, ohne formales Zustandsmodell. Mit React/Vue (ab 2013) entstand ein formales Client-Modell mit deklarativem Zustandsmanagement. Server-First-Frameworks (Next.js, SvelteKit, ab 2020) kehren zum Server zurück, aber mit Client-Hydration als Erweiterung. Die React-Komponentenidee hebt Separation of Concerns nicht auf — sie verschiebt sie von der Technologieachse (HTML/CSS/JS-Dateien) auf die Fachlichkeitsachse (Komponente: Produktkarte, Warenkorb, Navigation). Beide Interpretationen sind gültig und spiegeln unterschiedliche Systemkomplexitäten wider.

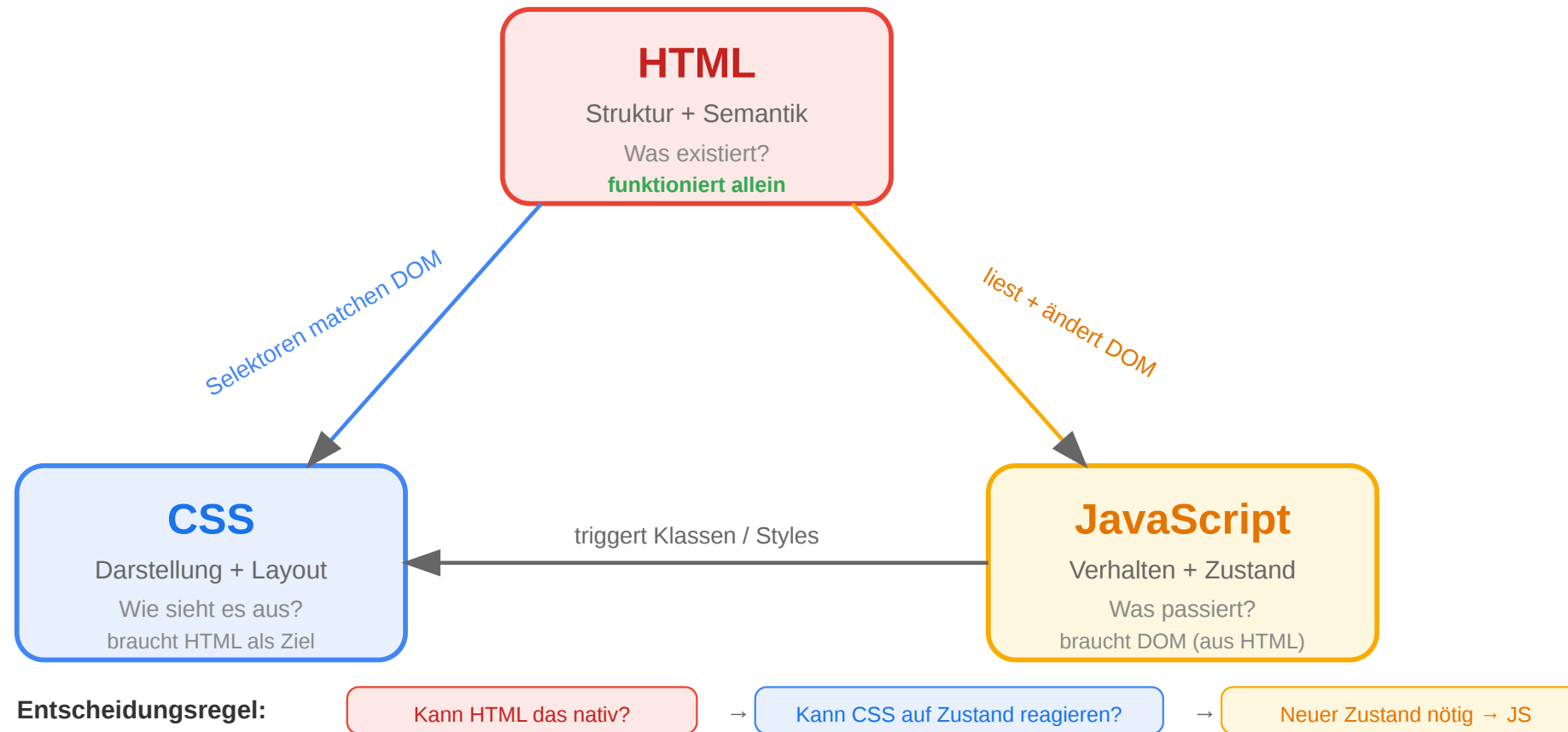
Einordnung

Beides stimmt: Die Trennung bleibt — aber auf einer anderen Ebene. Statt nach Technologie (HTML/CSS/JS) wird nach **fachlicher Verantwortung** getrennt (Produktkarte, Warenkorb, Navigation). Jede Komponente kapselt ihr Markup, ihren Stil und ihr Verhalten.

Separation of Concerns ist ein Prinzip, kein Mechanismus — es definiert das Ziel (verwandte Logik zusammen, unverwandte getrennt), nicht den Weg dahin. Was „zusammengehörig“ bedeutet, hängt von der Systemkomplexität ab. In dokumentartig aufgebauten Webseiten ist die Technologie die natürliche Trennlinie (HTML für Struktur, CSS für Darstellung, JS für Verhalten). Bei komplexen UI-Anwendungen wird die fachliche Verantwortung (Produktkarte, Warenkorb-Sidebar, Navigation) zur sinnvolleren Trennlinie — eine Komponente enthält HTML, CSS und JS, weil sie gemeinsam eine Fachlichkeit implementieren. Flexbox trennt CSS-Layoutlogik von HTML-Struktur auf Technologieebene, während Komponenten HTML+CSS+JS auf Fachebene zusammenfassen. Beide Ebenen koexistieren in modernen Frontends.

Das Technologie-Dreieck: Zusammenfassung

Das Technologie-Dreieck des Webs



HTML, CSS und JavaScript sind ein eng verzahntes System. Der Browser verarbeitet sie in einer festen Pipeline, die JavaScript jederzeit beeinflussen kann — mit Konsequenzen für Performance und Architektur. Moderne Frameworks organisieren das Zusammenspiel in Komponenten statt in Dateien. Das grundlegende Modell von DOM, Events und Rendering bleibt dasselbe.

Die Drei-Schichten-Architektur von HTML (Struktur), CSS (Darstellung) und JavaScript (Verhalten) ist stabiler als jedes konkrete Framework. React, Vue, Angular und Svelte haben alle dieselbe Grundstruktur: Sie erzeugen DOM (HTML), manipulieren Styles (CSS) und reagieren auf Events (JavaScript). Der Abstraktionsgrad ändert sich mit jeder Framework-Generation, das zugrunde liegende Browser-Modell (DOM, CSSOM, Event Loop, Rendering-Pipeline) ist seit über einem Jahrzehnt stabil. Ein Verständnis dieser Pipeline erklärt, warum bestimmte Framework-Optimierungen notwendig sind — Virtual DOM Diffing minimiert Reflows, CSS-in-JS scoped Styles auf Komponenten, Server-Side Rendering löst das SEO-Problem clientseitiger Apps.

Wrap-Up

- **HTML** definiert Struktur und Semantik — die Grundlage für Barrierefreiheit, SEO und maschinelle Verarbeitung.
- **JavaScript** macht den Browser programmierbar — über DOM, Events und asynchrone Kommunikation mit dem Server.
- **CSS** trennt Darstellung von Struktur — Flexbox und Grid lösen das Layout-Problem, Responsive Design das Geräteproblem.
- Die drei Technologien sind **eng verzahnt**: Der Browser verarbeitet sie in einer Pipeline (Parse → Render Tree → Layout → Paint), die JavaScript jederzeit beeinflussen kann.

Die nächste Vorlesung vertieft das **Protokoll** (HTTP) und das **API-Design** (REST), das die Brücke zwischen Frontend und Backend bildet.

HTML, CSS und JavaScript sind jeweils eigene Spezifikationen mit separaten Gremien (WHATWG, CSS Working Group, TC39) und separaten Releases — aber im Browser laufen sie in einer gemeinsamen Pipeline. Das konzeptionelle Modell dieser Vorlesung — HTML als Strukturschicht, CSS als Constraint-Solver, JavaScript als Event-gesteuerte Laufzeit — ist unabhängig vom Framework-Ökosystem gültig und überdauert Framework-Wechsel. Vorlesung 04 wechselt auf die Protokollebene: HTTP definiert, wie Browser und Server kommunizieren; REST und andere API-Stile definieren, wie diese Kommunikation semantisch strukturiert wird. Die Fetch API, die in dieser Vorlesung auf der Client-Seite betrachtet wurde, ist die JavaScript-Schnittstelle zu diesem Protokoll.

Legacy-Quellen:

- `slides/04_02_WebEng_WebTechnologies-HTML-5.pptx`
- `slides/04_03_WebEng_WebTechnologies-CSS3.pptx`
- JavaScript-Inhalte sind neu erstellt (kein Legacy-Material vorhanden)

